Sagar Rajen Kapadia

Automated Ontological Analysis

A Novel Framework for Rigorous Analysis of Ontologies

October 2025

Abstract

This book develops a comprehensive framework for *automated ontological analysis*, combining classical and modern methods from mathematics, logic, topology, and data science into a systematic approach. The guiding aim is to enable reproducible, quantitative, and computationally assisted evaluation of ontological structures, particularly in contexts where classification, categorization, and taxonomic rigor are foundational to knowledge organization.

The methodology integrates several dimensions:

- **Set-theoretic analysis**, including Jaccard similarity and pairwise intersection counts [1].
- **Topological diagnostics**, such as Betti curves and persistent homology on co-occurrence graphs [3].
- **Statistical methods**, including Principal Component Analysis (PCA) for dimensionality reduction and category emergence [4].
- **Graph-theoretic approaches**, including community detection and modularity analysis [10, 5].

Each method is applied at both the element level and the set level, yielding a multi-resolution perspective that clarifies redundancy, overlap, and independence within ontological state spaces. By comparing results across levels of analysis, the framework highlights where ontological coherence holds, where it collapses, and how higher-order conceptual structures emerge.

The contribution of this book lies not in the rescue or reform of any single ontology, but in providing a replicable architecture for *ontological autopsy*: the systematic evaluation of classification systems, exposing their strengths, weaknesses, and limits. The approach serves as both a diagnostic tool for existing knowledge systems and a generative foundation for the construction of more robust, logically consistent taxonomies in science and philosophy.

Contents

1 Introduction . . . 17
1.1 Motivation . . . 17
1.2 Scope and Methodology . . . 17
1.3 Structure of the Work . . . 17

References . . . 19

2 Techniques by Domain . . . 21

References . . . 23

Volume I: Information Theory

Part I Redundancy Ratio

3 Redundancy Ratio (General) . . . 29
3.1 Introduction . . . 29
3.2 Historical Perspective . . . 29
3.3 Conceptual Foundations . . . 29
3.4 Mathematical Formalism . . . 29
3.5 Pseudocode . . . 30
3.6 Python Algorithm . . . 30
3.7 Analysis and Interpretation . . . 30
3.8 Significance . . . 30
3.9 Conclusion . . . 30

References . . . 31

4 Redundancy Ratio: Element-Level Analysis . . . 33
4.1 Introduction . . . 33
4.2 Historical Perspective . . . 33
4.3 Conceptual Foundations . . . 33
4.4 Mathematical Formalism . . . 33
4.5 Pseudocode . . . 34
4.6 Python Algorithm . . . 34
4.7 Analysis . . . 34
4.8 Significance . . . 34
4.9 Conclusion . . . 34

References . . . 35

5 Redundancy Ratio: Set-Level Analysis . . . 37
5.1 Introduction . . . 37
5.2 Historical Perspective . . . 37
5.3 Conceptual Foundations . . . 37

5.4 Mathematical Formalism . . . 37
5.5 Pseudocode . . . 38
5.6 Python Algorithm . . . 38
5.7 Analysis . . . 38
5.8 Significance . . . 38
5.9 Conclusion . . . 38

References . . . 39

6 Redundancy Ratio: Interrelationships Between Element-Level and Set-Level . . . 41
6.1 Introduction . . . 41
6.2 Historical Perspective . . . 41
6.3 Conceptual Foundations . . . 41
6.4 Mathematical Relationship . . . 41
6.5 Analysis and Interpretation . . . 41
6.6 Resolution . . . 42
6.7 Significance . . . 42
6.8 Conclusion . . . 42

References . . . 43

Part II Ambiguous Elements Matrix

7 Ambiguous Elements (General) . . . 47
7.1 Introduction . . . 47
7.2 Historical Perspective . . . 47
7.3 Conceptual Foundations . . . 47
7.4 Mathematical Formalism . . . 47
7.5 Pseudocode . . . 48
7.6 Python Algorithm . . . 48
7.7 Analysis and Interpretation . . . 48
7.8 Significance . . . 48
7.9 Conclusion . . . 48

References . . . 49

8 Ambiguous Elements: Element-Level Analysis . . . 51
8.1 Introduction . . . 51
8.2 Historical Perspective . . . 51
8.3 Conceptual Foundations . . . 51
8.4 Pseudocode . . . 51
8.5 Python Algorithm . . . 52
8.6 Analysis . . . 52
8.7 Significance . . . 52
8.8 Conclusion . . . 52

References . . . 53

9 Ambiguous Elements: Set-Level Analysis . . . 55
9.1 Introduction . . . 55
9.2 Historical Perspective . . . 55
9.3 Conceptual Foundations . . . 55
9.4 Pseudocode . . . 55
9.5 Python Algorithm . . . 56
9.6 Analysis . . . 56
9.7 Significance . . . 56
9.8 Conclusion . . . 56

References . . . 57

10 Ambiguous Elements: Interrelationships Between Element-Level and Set-Level ... 59
10.1 Introduction ... 59
10.2 Historical Perspective ... 59
10.3 Conceptual Foundations ... 59
10.4 Analysis ... 59
10.5 Resolution ... 59
10.6 Conclusion ... 59

References ... 61

Volume II: Set Theory and Mathematics

Part III Ontological Set Mapping

11 Ontological Sphere Mapping: A Geometric Method for Visualizing Relations Between Sets ... 67
11.1 Introduction ... 67
11.2 Method ... 67
11.2.1 Representation of Sets ... 67
11.2.2 Intersection Volume ... 67
11.2.3 Axes of Embedding ... 68
11.3 Comparison to Existing Methods ... 68
11.4 Case Study: Psychiatric Nosology ... 68
11.5 General Applications ... 68
11.6 Conclusion ... 69

References ... 71

Part IV Pairwise-Intersection Analysis

12 Pairwise Intersection Counts Heatmap ... 75
12.1 Historical perspective ... 75
12.2 Mathematical formalism ... 75
12.3 Pseudocode ... 75
12.4 Python implementation ... 76
12.5 Analysis ... 76
12.6 Interpretation ... 76

References ... 77

13 Element-Level Intersection Analysis ... 79
13.1 Historical perspective ... 79
13.2 Mathematical formalism ... 79
13.3 Pseudocode ... 79
13.4 Python implementation ... 80
13.5 Analysis ... 80
13.6 Interpretation ... 80

References ... 81

14 Set-Level Intersection Analysis ... 83
14.1 Historical perspective ... 83
14.2 Mathematical formalism ... 83
14.3 Pseudocode ... 83
14.4 Python implementation ... 84
14.5 Analysis ... 84
14.6 Interpretation ... 84

References ... 85

15 Intersection Heatmaps: Element-Level vs. Set-Level Analysis . . . 87
15.1 Conceptual foundations . . . 87
15.2 Mathematical formalism . . . 87
15.3 Pseudocode . . . 87
15.3.1 Element-level . . . 87
15.3.2 Set-level . . . 88
15.4 Python implementation . . . 88
15.5 Analysis . . . 88
15.6 Resolution: Interpreting Differences . . . 89

References . . . 91

Part V Jacquard Analysis

16 The Jaccard Heatmap in Ontological Analysis . . . 95
16.1 Introduction . . . 95
16.2 Historical Context . . . 95
16.3 Conceptual Foundations . . . 95
16.4 Abstract Algorithm . . . 96
16.5 Python Implementation . . . 96
16.6 Interpretation of Values . . . 96
16.7 Significance in Ontological Analysis . . . 96
16.8 Conclusion . . . 97

References . . . 99

17 Jaccard Element-Level Analysis . . . 101
17.1 Mathematical formalism . . . 101
17.2 Pseudocode . . . 101
17.3 Python implementation . . . 102
17.4 Interpretation . . . 102

18 Jaccard Element-Level Analysis . . . 103
18.1 Mathematical formalism . . . 103
18.2 Pseudocode . . . 104
18.3 Python implementation . . . 104
18.4 Interpretation . . . 104
18.5 Conclusion . . . 105

19 Relating Element-Level and Set-Level Jaccard Analysis . . . 107
19.1 Conceptual foundations . . . 107
19.2 Mathematical relationship . . . 107
19.3 Pseudocode . . . 108
19.3.1 Element-level Jaccard . . . 108
19.3.2 Set-level Jaccard . . . 108
19.4 Python implementation . . . 109
19.5 Analysis . . . 109
19.6 Resolution: Interpreting Differences . . . 109

References . . . 111

20 Element-Level Jaccard Analysis vs Set-Level Jaccard Analysis . . . 113
20.1 Overview . . . 113
20.2 Mathematical formalism . . . 113
20.3 Set-level Jaccard . . . 113
20.4 Element-level Jaccard . . . 114
20.5 Key conceptual distinction . . . 114
20.6 Pseudocode (abstract) . . . 114
20.7 Python implementations . . . 115

20.8 Complexity and implementation notes . . . 115
20.9 Interpretation guidance . . . 116
20.10 Practical recommendations . . . 116

Volume III: Topology and Statistics

Part VI Betti Curves

21 Persistent Homology and Betti Curves: Foundations . . . 121
21.1 Historical perspective . . . 121
21.2 Mathematical formalism . . . 121
21.3 Pseudocode . . . 121
21.4 Python implementation . . . 122
21.5 Analysis . . . 122
21.6 Interpretation . . . 122

References . . . 123

22 Betti Curves on Ontology Overlap Graph (Element-Level) . . . 125
22.1 Historical perspective . . . 125
22.2 Mathematical formalism . . . 125
22.3 Pseudocode . . . 126
22.4 Python implementation . . . 126
22.5 Analysis . . . 127
22.6 Interpretation . . . 127

References . . . 129

23 Betti Curves on Ontology Overlap Graph (Set-Level) . . . 131
23.1 Historical perspective . . . 131
23.2 Mathematical formalism . . . 131
23.3 Pseudocode . . . 132
23.4 Python implementation . . . 132
23.5 Analysis . . . 133
23.6 Interpretation . . . 133

References . . . 135

24 Relating Set-Level and Element-Level Betti Curves . . . 137
24.1 Conceptual foundations . . . 137
24.2 Mathematical relationship . . . 137
24.3 Pseudocode for comparative analysis . . . 138
24.4 Python illustration . . . 138
24.5 Analysis . . . 138
24.6 Resolution . . . 138

References . . . 139

Part VII Persistent Homology

25 Persistent Homology (General) . . . 143
25.1 Introduction . . . 143
25.2 Historical Perspective . . . 143
25.3 Conceptual Foundations . . . 143
25.4 Mathematical Formalism . . . 143
25.5 Pseudocode . . . 144
25.6 Python Algorithm . . . 144
25.7 Analysis and Interpretation . . . 144
25.8 Significance . . . 144

25.9 Conclusion . . . 144

References . . . 145

26 Persistent Homology: Element-Level Analysis . . . 147
26.1 Introduction . . . 147
26.2 Historical Perspective . . . 147
26.3 Conceptual Foundations . . . 147
26.4 Mathematical Formalism . . . 147
26.5 Pseudocode . . . 148
26.6 Python Algorithm . . . 148
26.7 Analysis . . . 148
26.8 Significance . . . 148
26.9 Conclusion . . . 148

References . . . 149

27 Persistent Homology: Set-Level Analysis . . . 151
27.1 Introduction . . . 151
27.2 Historical Perspective . . . 151
27.3 Conceptual Foundations . . . 151
27.4 Mathematical Formalism . . . 151
27.5 Pseudocode . . . 151
27.6 Python Algorithm . . . 152
27.7 Analysis . . . 152
27.8 Significance . . . 152
27.9 Conclusion . . . 152

References . . . 153

28 Persistent Homology: Interrelationships Between Element-Level and Set-Level . . . 155
28.1 Introduction . . . 155
28.2 Historical Perspective . . . 155
28.3 Conceptual Foundations . . . 155
28.4 Analysis . . . 155
28.5 Resolution . . . 155
28.6 Conclusion . . . 156

References . . . 157

Volume IV: Statistics - Multivariate Analysis

Part VIII Axis Independence Score

29 Axis Independence Score (General) . . . 163
29.1 Introduction . . . 163
29.2 Historical Perspective . . . 163
29.3 Conceptual Foundations . . . 163
29.4 Mathematical Formalism . . . 163
29.5 Pseudocode . . . 164
29.6 Python Algorithm . . . 164
29.7 Analysis and Interpretation . . . 164
29.8 Significance . . . 164
29.9 Conclusion . . . 164

References . . . 165

30 Axis Independence Score: Element-Level Analysis . . . 167
30.1 Introduction . . . 167
30.2 Historical Perspective . . . 167
30.3 Conceptual Foundations . . . 167
30.4 Mathematical Formalism . . . 167
30.5 Pseudocode . . . 168
30.6 Python Algorithm . . . 168
30.7 Analysis . . . 168
30.8 Significance . . . 168
30.9 Conclusion . . . 168

References . . . 169

31 Axis Independence Score: Set-Level Analysis . . . 171
31.1 Introduction . . . 171
31.2 Historical Perspective . . . 171
31.3 Conceptual Foundations . . . 171
31.4 Mathematical Formalism . . . 171
31.5 Pseudocode . . . 171
31.6 Python Algorithm . . . 172
31.7 Analysis . . . 172
31.8 Significance . . . 172
31.9 Conclusion . . . 172

References . . . 173

32 Axis Independence Score: Interrelationships Between Element-Level and Set-Level 175
32.1 Introduction . . . 175
32.2 Historical Perspective . . . 175
32.3 Conceptual Foundations . . . 175
32.4 Analysis . . . 175
32.5 Resolution . . . 175
32.6 Conclusion . . . 176

References . . . 177

Part IX Principal Component Analysis

33 Principal Component Analysis: Foundations . . . 181
33.1 Historical perspective . . . 181
33.2 Mathematical formalism . . . 181
33.3 Pseudocode . . . 181
33.4 Python code . . . 182
33.5 Interpretation . . . 182

References . . . 183

34 Element-Level PCA . . . 185
34.1 Mathematical formulation . . . 185
34.2 Pseudocode . . . 185
34.3 Python implementation . . . 186
34.4 Analysis . . . 186
34.5 Interpretation . . . 186

References . . . 187

35 Set-Level PCA . . . 189
35.1 Mathematical formulation . . . 189
35.2 Pseudocode . . . 189
35.3 Python implementation . . . 190
35.4 Analysis . . . 190
35.5 Interpretation . . . 190

References . . . 191

36 Relating Element-Level and Set-Level PCA . . . 193
36.1 Conceptual foundations . . . 193
36.2 Mathematical relationship . . . 193
36.3 Pseudocode . . . 193
36.4 Python illustration . . . 194
36.5 Analysis . . . 194
36.6 Resolution . . . 194

References . . . 195

Part X Principal Component Analysis - The Scree Test

37 PCA Scree Analysis . . . 199
37.1 Historical perspective . . . 199
37.2 Conceptual foundations . . . 199
37.3 Mathematical formulation . . . 199
37.4 Pseudocode . . . 200
37.5 Python illustration . . . 200
37.6 Analysis and interpretation . . . 200
37.7 Conclusion . . . 200

References . . . 201

38 Element-Level PCA Scree Analysis . . . 203
38.1 Historical perspective . . . 203
38.2 Mathematical formulation . . . 203
38.3 Pseudocode . . . 203
38.4 Python implementation . . . 204
38.5 Analysis and interpretation . . . 204

References . . . 205

39 Set-Level PCA Scree Analysis . . . 207
39.1 Historical perspective . . . 207
39.2 Mathematical formulation . . . 207
39.3 Pseudocode . . . 207
39.4 Python implementation . . . 208
39.5 Analysis and interpretation . . . 208

References . . . 209

40 Interrelationships: Element vs. Set PCA Scree Analysis . . . 211
40.1 Historical perspective . . . 211
40.2 Mathematical relationship . . . 211
40.3 Pseudocode . . . 211
40.4 Python illustration . . . 212
40.5 Resolution . . . 212

References . . . 213

41 Interrelationships: PCA and Scree Analysis . . . 215
41.1 Historical perspective . . . 215
41.2 Conceptual relationship . . . 215
41.3 Mathematical linkage . . . 215
41.4 Pseudocode . . . 216
41.5 Python illustration . . . 216
41.6 Analysis . . . 216
41.7 Resolution . . . 216

References . . . 217

Part XI Principal Component Analysis - Loadings on Components

42 Principal Component Loadings . . . 221
42.1 Historical perspective . . . 221
42.2 Conceptual foundations . . . 221
42.3 Mathematical formulation . . . 221
42.4 Pseudocode . . . 222
42.5 Python illustration . . . 222
42.6 Analysis and interpretation . . . 222
42.7 Conclusion . . . 222

References . . . 223

43 Element-Level PCA Loadings . . . 225
43.1 Historical perspective . . . 225
43.2 Mathematical formulation . . . 225
43.3 Pseudocode . . . 225
43.4 Python illustration . . . 226
43.5 Analysis and interpretation . . . 226

References . . . 227

44 Set-Level PCA Loadings . . . 229
44.1 Historical perspective . . . 229
44.2 Mathematical formulation . . . 229
44.3 Pseudocode . . . 229
44.4 Python illustration . . . 230
44.5 Analysis and interpretation . . . 230

References . . . 231

45 Interrelationships: Element vs. Set PCA Loadings . . . 233
45.1 Historical perspective . . . 233
45.2 Conceptual relationship . . . 233
45.3 Mathematical linkage . . . 233
45.4 Pseudocode . . . 233
45.5 Python illustration . . . 234
45.6 Resolution . . . 234

References . . . 235

46 Interrelationships: PCA and Loadings . . . 237
46.1 Historical perspective . . . 237
46.2 Conceptual relationship . . . 237
46.3 Mathematical linkage . . . 237
46.4 Pseudocode . . . 238
46.5 Python illustration . . . 238
46.6 Analysis . . . 238
46.7 Resolution . . . 238

References 239

Volume V: Information Retrieval / Library Science

Part XII Overlap Index

47 Overlap Index (General) 245
47.1 Introduction 245
47.2 Historical Perspective 245
47.3 Conceptual Foundations 245
47.4 Mathematical Formalism 245
47.5 Pseudocode 246
47.6 Python Algorithm 246
47.7 Analysis and Interpretation 246
47.8 Significance 246
47.9 Conclusion 246

References 247

48 Overlap Index: Element-Level Analysis 249
48.1 Introduction 249
48.2 Historical Perspective 249
48.3 Conceptual Foundations 249
48.4 Mathematical Formalism 249
48.5 Pseudocode 250
48.6 Python Algorithm 250
48.7 Analysis 250
48.8 Significance 250
48.9 Conclusion 250

References 251

49 Overlap Index: Set-Level Analysis 253
49.1 Introduction 253
49.2 Historical Perspective 253
49.3 Conceptual Foundations 253
49.4 Mathematical Formalism 253
49.5 Pseudocode 254
49.6 Python Algorithm 254
49.7 Analysis 254
49.8 Significance 254
49.9 Conclusion 254

References 255

50 Overlap Index: Interrelationships Between Element-Level and Set-Level 257
50.1 Introduction 257
50.2 Historical Perspective 257
50.3 Conceptual Foundations 257
50.4 Analysis and Interpretation 257
50.5 Resolution 257
50.6 Conclusion 258

References 259

Part XIII Membership Histogram Analysis

51 Interrelationships: Element vs. Set Membership Histograms . . . 263
51.1 Conceptual relationship . . . 263
51.2 Resolution . . . 263

References . . . 265

52 Element-Level Membership Histogram . . . 267
52.1 Mathematical formulation . . . 267
52.2 Pseudocode . . . 267
52.3 Python illustration . . . 267
52.4 Analysis . . . 267

References . . . 269

53 Set-Level Membership Histogram . . . 271
53.1 Mathematical formulation . . . 271
53.2 Pseudocode . . . 271
53.3 Python illustration . . . 271
53.4 Analysis . . . 271

References . . . 273

54 Membership Histogram . . . 275
54.1 Historical perspective . . . 275
54.2 Conceptual foundations . . . 275
54.3 Pseudocode . . . 275
54.4 Python illustration . . . 276
54.5 Analysis . . . 276

References . . . 277

Volume VI: Network Science / Sociology

Part XIV Community Detection Analysis

55 Community Detection in Ontology Graphs . . . 283
55.1 Historical perspective . . . 283
55.2 Conceptual foundations . . . 283
55.3 Pseudocode . . . 283
55.4 Python illustration . . . 284
55.5 Analysis . . . 284

56 Element-Level Community Analysis . . . 285
56.1 Mathematical formulation . . . 285
56.2 Pseudocode . . . 285
56.3 Python illustration . . . 285
56.4 Analysis . . . 286

57 Set-Level Community Analysis . . . 287
57.1 Mathematical formulation . . . 287
57.2 Pseudocode . . . 287
57.3 Python illustration . . . 287
57.4 Analysis . . . 288

58 Interrelationships: Element vs. Set Communities . . . 289
58.1 Conceptual relationship . . . 289
58.2 Resolution . . . 289

Volume VII: Ontology Engineering / Philosophy of Science

Part XV Taxonomic Collapse Index

59 Taxonomic Collapse Index (General) . . . 295
59.1 Introduction . . . 295
59.2 Historical Perspective . . . 295
59.3 Conceptual Foundations . . . 295
59.4 Mathematical Formalism . . . 295
59.5 Pseudocode . . . 296
59.6 Python Algorithm . . . 296
59.7 Analysis and Interpretation . . . 296
59.8 Significance . . . 296
59.9 Conclusion . . . 296

References . . . 297

60 Taxonomic Collapse Index: Element-Level Analysis . . . 299
60.1 Introduction . . . 299
60.2 Historical Perspective . . . 299
60.3 Conceptual Foundations . . . 299
60.4 Mathematical Formalism . . . 299
60.5 Pseudocode . . . 299
60.6 Python Algorithm . . . 300
60.7 Analysis . . . 300
60.8 Significance . . . 300
60.9 Conclusion . . . 300

References . . . 301

61 Taxonomic Collapse Index: Set-Level Analysis . . . 303
61.1 Introduction . . . 303
61.2 Historical Perspective . . . 303
61.3 Conceptual Foundations . . . 303
61.4 Mathematical Formalism . . . 303
61.5 Pseudocode . . . 303
61.6 Python Algorithm . . . 304
61.7 Analysis . . . 304
61.8 Significance . . . 304
61.9 Conclusion . . . 304

References . . . 305

62 Taxonomic Collapse Index: Interrelationships Between Element-Level and Set-Level . . . 307
62.1 Introduction . . . 307
62.2 Historical Perspective . . . 307
62.3 Conceptual Foundations . . . 307
62.4 Analysis . . . 307
62.5 Resolution . . . 307
62.6 Conclusion . . . 308

References . . . 309

Part XVI Conclusion

63 Conclusion: Integrating Ontological Diagnostics . . . 313
63.1 Synthesis of diagnostics . . . 313
63.2 Complementarity . . . 313
63.3 Resolution of disagreements . . . 314
63.4 Conclusion . . . 314

References 315

A Appendix: Master Synthesis Table 317

B Appendix: Diagnostic Network Map 319

Chapter 1
Introduction

Ontologies provide a structured representation of domains, yet their complexity often obscures patterns of overlap, redundancy, and independence. Automated ontological analysis aims to make these patterns explicit through formal methods. This project presents a multi-method framework grounded in mathematics, statistics, and computer science.

1.1 Motivation

Ontologies are increasingly employed in knowledge representation, semantic web applications, and data integration. However, without systematic evaluation, they risk becoming inconsistent or redundant [1]. Manual inspection is infeasible for large-scale ontologies, motivating the need for automated approaches.

1.2 Scope and Methodology

The methodology integrates:

- **Set-theoretic measures:** Jaccard similarity [1] and pairwise intersections.
- **Topological methods:** Persistent homology and Betti curves [3].
- **Statistical analysis:** PCA for dimensionality reduction [4].

Each is implemented at both element-level and set-level, enabling fine-grained and global perspectives.

1.3 Structure of the Work

The chapters proceed thematically:

1. Jaccard analysis (heatmaps, element vs. set-level distinctions).
2. Pairwise intersection counts and their implications.
3. Betti curves and topological structure.
4. PCA and scree/loadings analysis.
5. Membership histograms and community detection.
6. Integrative discussion tying all components together.

Through this progression, the framework demonstrates how quantitative analysis uncovers the architecture of ontologies, providing both diagnostic insight and conceptual clarity.

References

References

[1] Jaccard, P. (1901). Étude comparative de la distribution florale. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies* (pp. 1–17). Springer.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer.

Chapter 2
Techniques by Domain

Abstract

This chapter presents a unified table of all techniques used in automated ontological analysis. Each technique is mapped to its disciplinary domain, with corresponding references. This structure allows quick navigation of methodological foundations and highlights the interdisciplinary lineage of ontology evaluation.

Sr#	Domain	Part (Technique)	Citations
1	Information Theory	Redundancy Ratio (RR)	[1, 1]
2	Information Theory	Ambiguous Elements Metric	[2, 2, 2]
3	Set Theory / Mathematics	Pairwise Intersection Counts	[1, 14]
4	Set Theory / Mathematics	Jaccard Similarity (Element vs Set)	[1]
5	Topology / TDA	Betti Curves (Threshold Filtrations)	[1, 1]
6	Topology / TDA	Persistent Homology	[3]
7	Statistics / Multivariate Analysis	Axis Independence Score (AIS)	[1, 1]
8	Statistics / Multivariate Analysis	PCA Scree Analysis	[4]
9	Statistics / Multivariate Analysis	PCA Loadings on Components	[4]
10	Information Retrieval / Library Science	Overlap Index (OI)	[1, 3, 1]
11	Information Retrieval / Library Science	Membership Histograms	[2]
12	Network Science / Sociology	Community Detection (Set Graph)	[10, 5]
13	Ontology Engineering / Philosophy of Science	Taxonomic Collapse Index (TCI)	[2, 1, 3]
14	Ontology Engineering / Philosophy of Science	Integrative Synthesis (Conclusion)	[1, 1]

Table 2.1 Summary of techniques grouped by domain with citations.

References

[1] Aquinas, T. (1274). *Summa Theologica*.

[1] Beghtol, C. (1986). Semantic validity: Concepts of warrant in bibliographic classification systems. *Library Resources & Technical Services*, 30(2), 109–125.

[3] Brank, J., Grobelnik, M., & Mladenić, D. (2005). A survey of ontology evaluation techniques. In *Proceedings of the Conference on Data Mining and Data Warehouses (SiKDD)*.

[1] Cantor, G. (1895). Beiträge zur Begründung der transfiniten Mengenlehre. *Mathematische Annalen*, 46, 481–512.

[1] Durkheim, E. (1895). *The Rules of Sociological Method*.

[1] Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2000). Topological persistence and simplification. *Proceedings 41st Annual Symposium on Foundations of Computer Science* (pp. 454–463). IEEE.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.

[1] Euler, L. (1736). Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, 8, 128–140.

[2] Garfield, E. (1979). *Citation Indexing: Its Theory and Application*. Wiley.

[10] Girvan, M., & Newman, M. E. J. (2002). Community structure in social and biological networks. *PNAS*, 99(12), 7821–7826.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

[1] Jaccard, P. (1901). Étude comparative de la distribution florale. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

[14] Kuratowski, C. (1921). Sur la notion de l'ordre de l'ensemble. *Fundamenta Mathematicae*, 2, 161–171.

[2] Kuhn, T. (1962). *The Structure of Scientific Revolutions*. University of Chicago Press.

[5] Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577–8582.

[1] Pearson, K. (1895). Notes on regression and inheritance. *Proceedings of the Royal Society of London*, 58, 240–242.

[1] Salton, G. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

[1] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

[1] Thurstone, L. L. (1935). *The Vectors of Mind*. University of Chicago Press.

[2] Zipf, G. K. (1945). The meaning-frequency relationship of words. *Science*, 101(2630), 409–411.

Volume I

Information Theory

Part I
Redundancy Ratio

Chapter 3
Redundancy Ratio (General)

Abstract

The Redundancy Ratio (RR) quantifies the degree to which elements in an ontology appear in multiple sets, thus measuring the reuse of components across categories. As a structural metric, RR provides a global index of redundancy and overlap, complementing measures such as Jaccard similarity and intersection counts. This chapter introduces the general framework for RR and situates it within the history of classification and ontology evaluation.

3.1 Introduction

Ontologies aim to provide clear boundaries between categories. However, redundancy—elements appearing in multiple sets—compromises clarity and independence. The Redundancy Ratio captures the overall burden of redundancy and provides a systematic diagnostic for ontological analysis.

3.2 Historical Perspective

Redundancy as a concept originates in information theory, where Shannon defined it as a measure of predictability or wasted coding capacity [1]. In taxonomy, naturalists such as Linnaeus sought to minimize redundancy by ensuring each species had a unique classification [2]. In ontology engineering, Guarino and colleagues emphasized the dangers of over-generalization and redundancy in conceptual models [1]. RR adapts this intellectual lineage to formal, automated ontology analysis.

3.3 Conceptual Foundations

Given sets $\{S_1, \ldots, S_n\}$ over universe U, RR measures how often elements appear redundantly across sets. A high RR signals weak ontological differentiation.

3.4 Mathematical Formalism

$$RR = \frac{1}{|U|} \sum_{e \in U} (m(e) - 1), \quad m(e) = |\{S_i : e \in S_i\}|$$

3.5 Pseudocode

```
Input: Sets S1, S2, ..., Sn
U = union of all sets
redundancy_sum = 0
For each element e in U:
    multiplicity = number of sets containing e
    redundancy_sum += (multiplicity - 1)
RR = redundancy_sum / |U|
Output RR
```

3.6 Python Algorithm

```python
def redundancy_ratio(ontology_sets):
    all_elements = set().union(*ontology_sets)
    redundancy_sum = 0
    for e in all_elements:
        multiplicity = sum(1 for S in ontology_sets if e in S)
        redundancy_sum += (multiplicity - 1)
    return redundancy_sum / len(all_elements) if all_elements else 0
```

3.7 Analysis and Interpretation

- $RR \approx 0$: clear, disjoint sets. - Moderate RR: partial overlap, manageable redundancy. - High RR: categories collapse into overused elements, loss of structure.

3.8 Significance

RR provides a global measure of how well or poorly ontological partitions are maintained. It is diagnostic of redundancy-driven collapse.

3.9 Conclusion

As a general index, RR is central to ontology autopsy: revealing redundancy as a structural failure mode.

References

References

[1] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[2] Linnaeus, C. (1758). *Systema Naturae*. Stockholm.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies* (pp. 1–17). Springer.

Chapter 4
Redundancy Ratio: Element-Level Analysis

Abstract

Element-level RR evaluates redundancy by counting how often individual elements recur across sets. This chapter formalizes the micro-level calculation and shows its diagnostic value.

4.1 Introduction

At the element level, RR highlights which specific items drive redundancy. This granularity reveals problematic concepts that cross too many categories.

4.2 Historical Perspective

The principle of single classification dates back to Aristotelian logic, where an entity was expected to belong to one genus and one differentia [1]. Violations of this principle were noted in medieval scholastic taxonomies. In modern information science, redundancy of terms across indexing categories was studied in thesaurus construction [2]. Element-level RR is a continuation of these concerns.

4.3 Conceptual Foundations

For each element $e \in U$, compute $m(e)$, the number of sets containing e. Redundancy is the surplus membership beyond one.

4.4 Mathematical Formalism

$$RR_{element} = \frac{1}{|U|} \sum_{e \in U} \max(0, m(e) - 1)$$

4.5 Pseudocode

```
For each element e in U:
   multiplicity = count of sets containing e
   redundancy += max(0, multiplicity-1)
RR_element = redundancy / |U|
```

4.6 Python Algorithm

```python
def element_level_rr(ontology_sets):
    U = set().union(*ontology_sets)
    redundancy = 0
    for e in U:
        multiplicity = sum(1 for S in ontology_sets if e in S)
        redundancy += max(0, multiplicity - 1)
    return redundancy / len(U) if U else 0
```

4.7 Analysis

- Elements with high multiplicity are ontological "overstretchers." - Distribution of multiplicities indicates whether redundancy is localized or systemic.

4.8 Significance

This lens identifies problematic elements, guiding refinement of ontology.

4.9 Conclusion

Element-level RR is diagnostic at micro scale, exposing specific redundancy drivers.

References

References

[1] Aristotle. (350 BCE). *Categories*.
[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

Chapter 5
Redundancy Ratio: Set-Level Analysis

Abstract

Set-level RR assesses redundancy at the category level, measuring how much categories overlap in their element composition. It reflects structural efficiency of sets as classifiers.

5.1 Introduction

While element-level analysis targets individual items, set-level RR evaluates categories as wholes. It indicates whether sets provide distinct partitions or collapse into overlapping clusters.

5.2 Historical Perspective

In classification history, libraries and encyclopedias attempted to create non-overlapping subject classes [1]. Redundant subject headings in library science motivated systematic measures of overlap [1]. Set-level RR continues this tradition.

5.3 Conceptual Foundations

Define for set S_i the redundancy contribution as the sum of excess overlaps relative to other sets.

5.4 Mathematical Formalism

$$RR_{set} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{1}{|S_i|} \sum_{e \in S_i} (m(e) - 1) \right)$$

5.5 Pseudocode

```
For each set S_i:
   set_redundancy = 0
   For each element e in S_i:
       multiplicity = number of sets containing e
       set_redundancy += (multiplicity - 1)
   set_RR_i = set_redundancy / |S_i|
RR_set = average of set_RR_i over all sets
```

5.6 Python Algorithm

```python
def set_level_rr(ontology_sets):
    rr_values = []
    for S in ontology_sets:
        set_redundancy = 0
        for e in S:
            multiplicity = sum(1 for T in ontology_sets if e in T)
            set_redundancy += (multiplicity - 1)
        rr_values.append(set_redundancy / len(S) if S else 0)
    return sum(rr_values)/len(rr_values) if rr_values else 0
```

5.7 Analysis

- Low RR: sets are efficient classifiers. - High RR: categories are structurally redundant.

5.8 Significance

Set-level RR diagnoses whether ontological categories carry independent classificatory power.

5.9 Conclusion

Set-level RR exposes redundancy at the meso-scale, highlighting inefficiencies in category construction.

References

References

[1] Otlet, P. (1934). *Traité de Documentation.*

[1] Beghtol, C. (1986). Semantic validity: Concepts of warrant in bibliographic classification systems. *Library Resources & Technical Services*, 30(2), 109–125.

Chapter 6
Redundancy Ratio: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter analyzes the relationship between element-level and set-level RR, emphasizing cases where the two perspectives diverge. A resolution framework is introduced.

6.1 Introduction

Element-level and set-level RR offer complementary perspectives. Their agreement signals robust measurement; their divergence signals complex redundancy patterns.

6.2 Historical Perspective

The tension between micro-level and macro-level classification has been noted since Durkheim's sociology, where individual and collective categories sometimes misalign [1]. In modern information science, micro-index terms vs. macro categories often reveal similar mismatches [2]. This chapter formalizes such mismatches for RR.

6.3 Conceptual Foundations

- Element-level RR identifies overstretched elements. - Set-level RR identifies structurally redundant categories. - Comparing them reveals whether redundancy is localized or systemic.

6.4 Mathematical Relationship

$$RR_{element} = \frac{1}{|U|} \sum_{e \in U} (m(e) - 1), \quad RR_{set} = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{|S_i|} \sum_{e \in S_i} (m(e) - 1)$$

6.5 Analysis and Interpretation

- If $RR_{element} \approx RR_{set}$: redundancy is evenly distributed. - If $RR_{element} \gg RR_{set}$: a few overstretched elements distort ontology. - If $RR_{set} \gg RR_{element}$: redundancy is structural, affecting categories globally.

6.6 Resolution

When values differ significantly, ontology analysts must distinguish between localized problematic elements and systemic redundancy in the set structure.

6.7 Significance

This comparative lens provides clarity on whether ontological failure is caused by specific elements or category construction.

6.8 Conclusion

RR interrelationships enable nuanced diagnosis, ensuring redundancy is understood at both micro and macro levels.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.
[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

Part II
Ambiguous Elements Matrix

Chapter 7
Ambiguous Elements (General)

Abstract

Ambiguous elements are those that occur in multiple categories of an ontology, thereby weakening the distinctness of taxonomic boundaries. This chapter introduces the concept of ambiguous elements, situates it historically, and develops the mathematical and algorithmic framework for their detection and evaluation.

7.1 Introduction

An ontology relies on clear boundaries between categories. Elements that appear in multiple sets blur these boundaries, generating ambiguity and redundancy. Quantifying ambiguous elements provides a diagnostic for structural clarity.

7.2 Historical Perspective

The problem of ambiguous terms dates back to Aristotle's logic, where equivocal terms disrupted categorical reasoning [1]. In linguistics, Zipf noted the ubiquity of polysemy, where words carried multiple meanings [2]. Bibliometrics later highlighted the role of "hub terms" that appeared across disciplines, muddying classification [2]. Ontology evaluation inherits this concern, treating ambiguous elements as measurable signals of taxonomic weakness.

7.3 Conceptual Foundations

Ambiguity arises when an element $e \in U$ belongs to more than one set in $\{S_1, \ldots, S_n\}$. The degree of ambiguity is proportional to the multiplicity of membership.

7.4 Mathematical Formalism

Define:

$$Amb(e) = m(e) - 1$$

where $m(e)$ is the number of sets containing e.

Aggregate ambiguity across the ontology:

$$Ambiguity\ Index = \frac{1}{|U|} \sum_{e \in U} Amb(e)$$

7.5 Pseudocode

```
Input: Sets S1,...,Sn
U = union of all sets
For each element e in U:
   multiplicity = number of sets containing e
   Amb(e) = multiplicity - 1
Ambiguity Index = average Amb(e)
```

7.6 Python Algorithm

```python
def ambiguity_index(ontology_sets):
    U = set().union(*ontology_sets)
    return sum(sum(1 for S in ontology_sets if e in S) - 1 for e in U) / len(U)
```

7.7 Analysis and Interpretation

- Amb(e) = 0 → element unique to one category. - Amb(e) ¿ 0 → element spans categories. - High Ambiguity Index → ontology structurally unclear.

7.8 Significance

Ambiguous elements highlight weak taxonomic distinctions and reveal collapse points.

7.9 Conclusion

Measuring ambiguous elements is crucial for diagnosing the clarity and resilience of ontologies.

References

References

[1] Aristotle. (350 BCE). *Categories*.
[2] Zipf, G. K. (1945). *The Meaning-Frequency Relationship of Words*. Science, 101(2630), 409–411.
[2] Garfield, E. (1979). *Citation Indexing: Its Theory and Application*. Wiley.

Chapter 8
Ambiguous Elements: Element-Level Analysis

Abstract

Element-level analysis identifies which specific elements are ambiguous, quantifying their individual contributions to ontological collapse. This chapter formalizes the micro-level detection of ambiguous elements.

8.1 Introduction

Some elements appear in many sets, serving as cross-cutting "bridges." These elements are identified through multiplicity counts.

8.2 Historical Perspective

The ambiguity of terms was a recurring issue in scholastic philosophy, where polysemous words destabilized categorical hierarchies. In modern information science, Soergel noted that thesauri often reused terms across unrelated domains [2]. Element-level ambiguity analysis continues this line of inquiry.

8.3 Conceptual Foundations

For element e:

$$Amb(e) = m(e) - 1$$

8.4 Pseudocode

```
For each element e in U:
   multiplicity = number of sets containing e
   Amb(e) = multiplicity - 1
Store Amb(e)
```

8.5 Python Algorithm

```
def element_level_ambiguity(ontology_sets):
    U = set().union(*ontology_sets)
    scores = {}
    for e in U:
        scores[e] = sum(1 for S in ontology_sets if e in S) - 1
    return scores
```

8.6 Analysis

- High Amb(e): element is highly ambiguous. - Amb(e) = 0: element is unique.

8.7 Significance

Element-level analysis pinpoints specific collapse drivers.

8.8 Conclusion

This perspective isolates ambiguous elements as focal points of ontological breakdown.

References

References

[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

Chapter 9
Ambiguous Elements: Set-Level Analysis

Abstract

Set-level analysis measures the distribution of ambiguous elements across sets, highlighting categories most affected by ambiguity.

9.1 Introduction

Not all sets are equally compromised by ambiguous elements. Some categories may be dominated by ambiguous items, undermining their validity.

9.2 Historical Perspective

Library science confronted this issue in subject indexing, where overly broad categories collected many ambiguous terms [1]. Ontology evaluation inherits this concern, asking whether sets are clearly distinguished or overwhelmed by ambiguity.

9.3 Conceptual Foundations

For set S_i:

$$Amb(S_i) = \frac{|\{e \in S_i : Amb(e) > 0\}|}{|S_i|}$$

9.4 Pseudocode

```
For each set Si:
   count ambiguous elements in Si
   Amb(Si) = count / |Si|
```

9.5 Python Algorithm

```
def set_level_ambiguity(ontology_sets):
    elem_scores = element_level_ambiguity(ontology_sets)
    results = []
    for S in ontology_sets:
        ambiguous_count = sum(1 for e in S if elem_scores[e] > 0)
        results.append(ambiguous_count / len(S) if S else 0)
    return results
```

9.6 Analysis

- High Amb(Si): set undermined by ambiguous terms. - Low Amb(Si): set structurally clear.

9.7 Significance

This reveals which categories are fragile and which are robust.

9.8 Conclusion

Set-level ambiguity analysis provides macro-level insight into structural clarity.

References

References

[1] Beghtol, C. (1986). Semantic validity: Concepts of warrant in bibliographic classification systems. *Library Resources & Technical Services*, 30(2), 109–125.

Chapter 10
Ambiguous Elements: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter compares element-level and set-level ambiguity, resolving discrepancies and clarifying whether collapse originates from specific elements or systemic category weakness.

10.1 Introduction

Micro- and macro-level perspectives may diverge: ambiguous elements may be concentrated in few sets or evenly distributed.

10.2 Historical Perspective

Durkheim emphasized the difference between individual ambiguity and structural collapse in classification of social facts [1]. Ontology analysis inherits this dual-level concern.

10.3 Conceptual Foundations

- Element-level: Ambiguity localized in terms. - Set-level: Ambiguity distributed across categories.

10.4 Analysis

- Both high: systemic collapse. - Element high, set low: few elements destabilize ontology. - Set high, element low: ambiguity systemic across categories.

10.5 Resolution

Comparison resolves whether refinement targets specific elements or restructuring of categories.

10.6 Conclusion

Ambiguity analysis across levels clarifies the structural health of the ontology.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.

Volume II

Set Theory and Mathematics

Part III
Ontological Set Mapping

Chapter 11
Ontological Sphere Mapping: A Geometric Method for Visualizing Relations Between Sets

Abstract

We introduce a novel visualization technique for representing relations between sets using a continuous 3D embedding of *spheres in semantic state space*. Each set is represented as a sphere whose radius reflects set size, while the volumetric intersection of spheres encodes overlap between sets. This approach generalizes Venn and Euler diagrams, which are limited to three sets, and provides quantitative, scalable, and interpretable visualizations of high-dimensional overlaps. We demonstrate the method using psychiatric nosology (DSM-5 disorders), where semantic axes correspond to ontological domains (Mood–Anxiety, Psychosis, Intrusions), and show how disorder overlaps can be geometrically quantified. We argue that *Ontological Sphere Mapping* offers a new paradigm for set visualization across science, medicine, and social systems.

11.1 Introduction

The visualization of overlapping sets is a long-standing challenge in information visualization. Traditional approaches such as Venn and Euler diagrams are expressive for up to three sets but scale poorly. Other methods (UpSet plots, network-based overlap graphs, matrix visualizations) represent overlap numerically but lose geometric intuition.

We propose a new technique, *Ontological Sphere Mapping (OSM)*, in which sets are represented as spheres embedded in a continuous metric space. Sphere radius is proportional to set size, while the volume of pairwise intersections reflects the degree of overlap. By anchoring the embedding in *semantic axes*, the method enables both quantitative analysis and interpretable visualization.

11.2 Method

11.2.1 Representation of Sets

Given a collection of sets $\{S_1, S_2, \ldots, S_n\}$, we define:

- Sphere center coordinates (x_i, y_i, z_i) in R^3, determined either by semantic dimensions (ontology-driven) or dimensionality reduction (MDS, PCA) of a similarity matrix.
- Sphere radius r_i proportional to $|S_i|$, i.e. $r_i = \alpha + \beta|S_i|$ for scaling constants α, β.
- Pairwise overlap measured by volumetric intersection $V(S_i, S_j)$, computable from analytic formulae for intersecting spheres.

11.2.2 Intersection Volume

For two spheres with radii r_1, r_2 and center distance d, the intersection volume is:

$$V = \begin{cases} 0, & d \geq r_1 + r_2 \\ \frac{4}{3}\pi \min(r_1, r_2)^3, & d \leq |r_1 - r_2| \\ \frac{\pi(r_1+r_2-d)^2\left(d^2+2d(r_1+r_2)-3(r_1-r_2)^2\right)}{12d}, & \text{otherwise.} \end{cases}$$

This enables calculation of both absolute overlap volume and normalized percentage overlap relative to the smaller set.

11.2.3 Axes of Embedding

Axes can be defined in two ways:

1. **Ontology-driven:** Using theoretically meaningful dimensions (e.g., psychiatric domains such as Mood, Psychosis, Intrusions).
2. **Data-driven:** Using similarity matrices (e.g., Jaccard overlap) embedded via MDS or PCA.

11.3 Comparison to Existing Methods

- **Venn / Euler diagrams:** Limited to 2–3 sets, lack scaling. OSM generalizes to n sets.
- **UpSet plots:** Quantitative but not geometric. OSM preserves both.
- **Network graphs:** Capture overlaps via edge weights, but lose volumetric intuition. OSM encodes overlap as geometric volume.

To our knowledge, no existing set visualization method uses volumetric intersections of spheres in semantically meaningful spaces, making OSM a novel contribution.

11.4 Case Study: Psychiatric Nosology

We applied OSM to visualize relations among DSM-5 psychiatric disorders: Major Depressive Disorder (MDD), Generalized Anxiety Disorder (GAD), Post-Traumatic Stress Disorder (PTSD), Mania, Schizophrenia, and Obsessive-Compulsive Disorder (OCD).

- **Axes:** Mood–Anxiety, Psychosis, Intrusions.
- **Sphere sizes:** Number of DSM-5 diagnostic domains.
- **Overlap:** Disorders with multiple shared domains (e.g., PTSD–Schizophrenia) exhibited large intersection volumes, visualizing clinical comorbidity.

This revealed three major spectra: (i) Mood–Anxiety, (ii) Psychosis, (iii) Intrusion, with PTSD as a hub overlapping all domains.

11.5 General Applications

OSM is generalizable to:

- Genomics: genes × pathways × phenotypes.
- Law: rights × statutes × jurisdictions.
- Sociology: ideologies × value systems × networks.
- Data science: overlapping feature sets, user groups, or clusters.

11.6 Conclusion

Ontological Sphere Mapping (OSM) provides a novel, scalable, and semantically interpretable way to visualize overlapping sets. By embedding sets as spheres in a continuous state space, OSM unifies geometry, ontology, and quantitative overlap analysis. We believe this technique opens new possibilities for exploratory visualization across scientific domains.

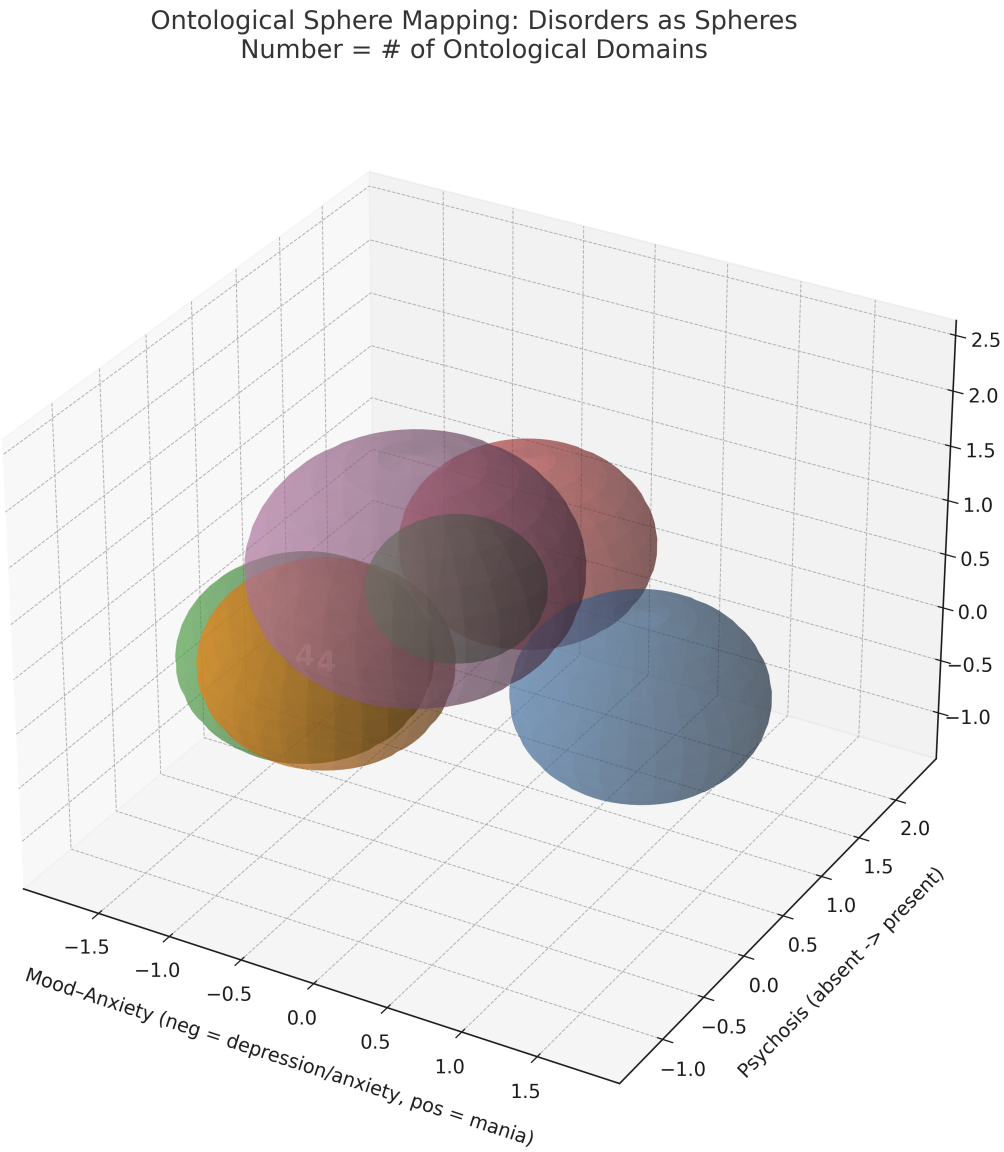

Fig. 11.1 Ontological Sphere Mapping of DSM-5 disorders. Each disorder is represented as a sphere in a 3D state-space defined by Mood–Anxiety, Psychosis, and Intrusion axes. Sphere radius is proportional to the number of ontological domains spanned by the disorder. Numbers inside spheres indicate the exact domain count. Overlaps between spheres represent shared ontological domains, with volumetric intersections reflecting the magnitude of overlap.

References

[1] Alsallakh, B., et al. "Visualizing sets and set-typed data: State-of-the-art and future challenges." *Eurographics Conference on Visualization (EuroVis)*, 2016.

[2] Collins, C., et al. "Bubble Sets: Revealing Set Relations with Isocontours over Existing Visualizations." *IEEE Transactions on Visualization and Computer Graphics*, 2009.

Part IV
Pairwise-Intersection Analysis

Chapter 12
Pairwise Intersection Counts Heatmap

Abstract

This chapter introduces the intersection-counts heatmap as a foundational diagnostic for ontology analysis. It visualizes the raw number of shared elements between pairs of sets, emphasizing absolute overlap rather than proportional similarity [1].

12.1 Historical perspective

The use of intersection counts dates back to classical set theory in the late 19th century with Cantor's work on cardinalities [1]. In information retrieval and early clustering analysis, raw overlap counts were among the first measures of similarity, preceding normalized indices such as the Jaccard index [1]. Intersection counts remain relevant in domains where absolute redundancy is more informative than proportional similarity.

12.2 Mathematical formalism

For two sets S_i, S_j:

$$I(S_i, S_j) = |S_i \cap S_j|.$$

Construct the symmetric matrix $I = [I(S_i, S_j)]_{i,j=1}^{n}$, with diagonal entries $I(S_i, S_i) = |S_i|$.

12.3 Pseudocode

```
\Procedure{ComputeIntersectionCounts}{Sets S[1..n]}
  for i = 1..n:
    for j = i..n:
      inter = |S[i]     S[j]|
      I[i,j] = inter
      I[j,i] = inter
  return I
\EndProcedure
```

12.4 Python implementation

```
def intersection_counts_heatmap(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    I = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            I[i,j] = len(ontology_sets[i] & ontology_sets[j])
    plt.imshow(I, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Intersection Count")
    plt.title("Pairwise Intersection Counts Heatmap")
    out_path = os.path.join(out_dir, "intersection_counts.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return I, out_path
```

12.5 Analysis

The intersection count matrix highlights the magnitude of shared elements across categories. Large diagonal values indicate the size of each category, while large off-diagonal values indicate redundancy. This measure is sensitive to category size, making it a blunt but revealing instrument for ontology diagnostics.

12.6 Interpretation

- High off-diagonal values: categories that overlap heavily.
- Low off-diagonal values: categories that remain distinct.
- As a general diagnostic, this measure sets the stage for refined analyses at the element and set levels.

References

[1] Cantor, G. (1895). *Beiträge zur Begründung der transfiniten Mengenlehre*. Mathematische Annalen, 46, 481–512.

[1] Jaccard, P. (1901). *Étude comparative de la distribution florale dans une portion des Alpes et du Jura*. Bulletin de la Société Vaudoise des Sciences Naturelles, 37, 547–579.

[1] Pearson, K. (1895). *Note on regression and inheritance in the case of two parents*. Proceedings of the Royal Society of London, 58, 240–242.

[2] Lund, K., & Burgess, C. (1996). *Producing high-dimensional semantic spaces from lexical co-occurrence*. Behavior Research Methods, Instruments, & Computers, 28, 203–208.

[3] Landauer, T. K., & Dumais, S. T. (1997). *A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge*. Psychological Review, 104(2), 211–240.

[1] Sørensen, T. (1948). *A method of establishing groups of equal amplitude in plant sociology based on similarity of species and its application to analyses of the vegetation on Danish commons*. Biologiske Skrifter, 5(4), 1–34.

Chapter 13
Element-Level Intersection Analysis

Abstract

This chapter applies intersection analysis at the element level, where elements are compared according to their co-occurrence across categories. This reveals which elements function as cross-cutting features across the ontology [1].

13.1 Historical perspective

Element co-occurrence analysis is foundational in statistics, beginning with Pearson's work on correlation [1], and later developed in text mining, where word co-occurrence matrices became a core tool in semantic similarity analysis [2, 3]. In ontology analysis, it provides insight into whether elements are category-specific or promiscuous.

13.2 Mathematical formalism

For element $e \in U$, define its profile:

$$P(e) = \{i \mid e \in S_i\}.$$

For two elements $e, f \in U$:

$$I_{\text{elem}}(e, f) = |P(e) \cap P(f)|.$$

13.3 Pseudocode

```
\Procedure{ComputeElementIntersection}{Sets S[1..n]}
  U = union of all elements
  for each element e in U:
    P[e] = { i | e     S[i] }
  for each pair (e,f):
    I_elem[e,f] = |P[e]     P[f]|
  return I_elem
\EndProcedure
```

13.4 Python implementation

```
def element_intersection_matrix(ontology_sets, out_dir="."):
    all_elements = sorted(set().union(*ontology_sets))
    profiles = {e: set() for e in all_elements}
    for si, s in enumerate(ontology_sets):
        for e in s: profiles[e].add(si)
    m = len(all_elements)
    I = np.zeros((m, m), dtype=int)
    for i,e1 in enumerate(all_elements):
        for j,e2 in enumerate(all_elements):
            I[i,j] = len(profiles[e1] & profiles[e2])
    plt.imshow(I, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Intersection Count (Elements)")
    plt.title("Element-Level Intersection Heatmap")
    out_path = os.path.join(out_dir, "element_intersections.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return I, all_elements, out_path
```

13.5 Analysis

The element-level intersection matrix captures how often elements appear together across categories. High co-occurrence signals redundancy: elements that are jointly assigned across categories, thus undermining ontological distinctiveness. Sparse rows or columns indicate unique elements.

13.6 Interpretation

- High I_{elem}: elements tightly coupled across many sets.
- Low I_{elem}: elements discriminative of specific sets.
- Useful for identifying elements that blur category boundaries.

References

[1] Cantor, G. (1895). *Beiträge zur Begründung der transfiniten Mengenlehre*. Mathematische Annalen, 46, 481–512.

[1] Jaccard, P. (1901). *Étude comparative de la distribution florale dans une portion des Alpes et du Jura*. Bulletin de la Société Vaudoise des Sciences Naturelles, 37, 547–579.

[1] Pearson, K. (1895). *Note on regression and inheritance in the case of two parents*. Proceedings of the Royal Society of London, 58, 240–242.

[2] Lund, K., & Burgess, C. (1996). *Producing high-dimensional semantic spaces from lexical co-occurrence*. Behavior Research Methods, Instruments, & Computers, 28, 203–208.

[3] Landauer, T. K., & Dumais, S. T. (1997). *A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge*. Psychological Review, 104(2), 211–240.

[1] Sørensen, T. (1948). *A method of establishing groups of equal amplitude in plant sociology based on similarity of species and its application to analyses of the vegetation on Danish commons*. Biologiske Skrifter, 5(4), 1–34.

Chapter 14
Set-Level Intersection Analysis

Abstract

This chapter applies intersection analysis at the set level, comparing categories directly by the number of elements they share. This measure diagnoses redundancy between sets in terms of their raw overlap [1].

14.1 Historical perspective

Direct set overlap measures have long been used in classification evaluation, such as the Sørensen–Dice coefficient [1]. In ontology contexts, they are the most immediate way to measure redundancy between categories.

14.2 Mathematical formalism

For sets S_i, S_j:

$$I_{\text{set}}(S_i, S_j) = |S_i \cap S_j|.$$

14.3 Pseudocode

```
\Procedure{ComputeSetIntersection}{Sets S[1..n]}
  for i = 1..n:
    for j = i..n:
      I_set[i,j] = |S[i]     S[j]|
      I_set[j,i] = I_set[i,j]
  return I_set
\EndProcedure
```

14.4 Python implementation

```
def set_intersection_matrix(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    I = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            I[i,j] = len(ontology_sets[i] & ontology_sets[j])
    plt.imshow(I, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Intersection Count (Sets)")
    plt.title("Set-Level Intersection Heatmap")
    out_path = os.path.join(out_dir, "set_intersections.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return I, out_path
```

14.5 Analysis

The set-level intersection matrix directly reveals category overlaps. High values signify redundancy, and diagonal dominance reflects set size. Unlike normalized measures, this preserves the scale of absolute overlap.

14.6 Interpretation

- $I_{\text{set}} = 0$: disjoint sets, distinct categories.
- Large I_{set}: categories overlap heavily, risking collapse.
- A mix of large and small overlaps reveals uneven redundancy patterns.

References

[1] Cantor, G. (1895). *Beiträge zur Begründung der transfiniten Mengenlehre*. Mathematische Annalen, 46, 481–512.

[1] Jaccard, P. (1901). *Étude comparative de la distribution florale dans une portion des Alpes et du Jura*. Bulletin de la Société Vaudoise des Sciences Naturelles, 37, 547–579.

[1] Pearson, K. (1895). *Note on regression and inheritance in the case of two parents*. Proceedings of the Royal Society of London, 58, 240–242.

[2] Lund, K., & Burgess, C. (1996). *Producing high-dimensional semantic spaces from lexical co-occurrence*. Behavior Research Methods, Instruments, & Computers, 28, 203–208.

[3] Landauer, T. K., & Dumais, S. T. (1997). *A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge*. Psychological Review, 104(2), 211–240.

[1] Sørensen, T. (1948). *A method of establishing groups of equal amplitude in plant sociology based on similarity of species and its application to analyses of the vegetation on Danish commons*. Biologiske Skrifter, 5(4), 1–34.

Chapter 15
Intersection Heatmaps: Element-Level vs. Set-Level Analysis

Abstract

This chapter explores the use of intersection heatmaps in ontology diagnostics, comparing the element-level and set-level projections. At the element level, the heatmap quantifies pairwise co-membership across sets; at the set level, it quantifies pairwise overlap in terms of shared elements. The comparison of these two views provides insights into whether redundancy lies in the assignments of elements or in the structuring of categories. A resolution section highlights how to interpret differences between these two projections [1, 1].

15.1 Conceptual foundations

As with Betti curve analysis, ontology structure can be probed at two levels:

- **Element-level intersection:** For each pair of elements (e, f), count the number of sets that contain both. This yields an $|U| \times |U|$ heatmap, where U is the element universe.
- **Set-level intersection:** For each pair of sets (S_i, S_j), count the number of shared elements. This yields an $n \times n$ heatmap, where n is the number of sets.

Both capture overlap, but at different scales.

15.2 Mathematical formalism

Let M be the incidence matrix, with $M_{ij} = 1$ if element $e_i \in S_j$.

$$A = MM^\top, \quad A_{ef} = |P(e) \cap P(f)|,$$
$$B = M^\top M, \quad B_{ij} = |S_i \cap S_j|.$$

Here, A produces the element-level heatmap, while B produces the set-level heatmap.

15.3 Pseudocode

15.3.1 Element-level

```
\Procedure{ElementIntersectionHeatmap}{Ontology sets S}
  Elements = union of all sets
  for each pair (e,f):
    A[e,f] = |{i | e,f     S[i]}|
  return A
\EndProcedure
```

15.3.2 Set-level

```
\Procedure{SetIntersectionHeatmap}{Ontology sets S}
  for each pair (Si,Sj):
    B[i,j] = |Si     Sj|
  return B
\EndProcedure
```

15.4 Python implementation

```python
import numpy as np
import matplotlib.pyplot as plt
import itertools, os

def element_intersection_heatmap(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    n = len(elements)
    A = np.zeros((n,n))
    for i, e1 in enumerate(elements):
        for j, e2 in enumerate(elements):
            if i <= j:
                val = sum(1 for s in ontology_sets if e1 in s and e2 in s)
                A[i,j] = A[j,i] = val
    plt.imshow(A, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Shared Sets")
    plt.title("Element Intersection Heatmap")
    out_path = os.path.join(out_dir,"intersection_elements.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return A, out_path

def set_intersection_heatmap(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    B = np.zeros((n,n))
    for i, Si in enumerate(ontology_sets):
        for j, Sj in enumerate(ontology_sets):
            if i <= j:
                val = len(Si & Sj)
                B[i,j] = B[j,i] = val
    plt.imshow(B, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Shared Elements")
    plt.title("Set Intersection Heatmap")
    out_path = os.path.join(out_dir,"intersection_sets.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return B, out_path
```

15.5 Analysis

- The element-level heatmap (A) reveals redundancy of elements across categories. High values indicate elements consistently co-occurring.
- The set-level heatmap (B) reveals redundancy among categories. High values indicate categories collapsing into indistinct clusters.

Both heatmaps are diagnostic of overlap, but they emphasize different failure modes.

15.6 Resolution: Interpreting Differences

- **High redundancy in A, low in B:** Elements are overused across categories, but categories remain somewhat distinct. Indicates semantic overloading of elements.
- **High redundancy in B, low in A:** Categories collapse due to overlap, but elements remain distributed. Indicates collapse of the classification scheme.
- **Both high:** Systemic redundancy at both levels — neither categories nor elements are coherent.
- **Both low:** Ontology is well-separated, minimal redundancy.

References

[1] Pearson, K. (1895). Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242.

[1] Sørensen, T. (1948). A method of establishing groups of equal amplitude in plant sociology based on similarity of species and its application to analyses of the vegetation on Danish commons. *Biologiske Skrifter*, 5(4), 1–34.

[1] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

[2] Ghrist, R. (2008). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

Part V
Jacquard Analysis

Chapter 16
The Jaccard Heatmap in Ontological Analysis

Abstract

This chapter examines the role of the Jaccard index in ontology evaluation. It develops the mathematical foundations of the index, traces its historical origins, distinguishes between element-level and set-level applications, and explains its implementation in automated analysis pipelines. A worked-out algorithm in both formal notation and Python pseudocode is provided. The chapter concludes with an interpretation of values, their diagnostic use in ontology validation, and the broader significance of Jaccard heatmaps in detecting redundancy and structural weaknesses in classificatory systems.

16.1 Introduction

Ontologies and taxonomies are intended to provide clear categorical boundaries. When categories overlap excessively, their epistemic credibility as distinct types is compromised. Quantifying this overlap is therefore essential to ontology analysis. The Jaccard index offers a historically established and mathematically rigorous measure of set similarity that has found broad application in ecology [1], statistics [1], ontology evaluation [1, 3], and information retrieval [5].

16.2 Historical Context

The Jaccard index was introduced by Paul Jaccard in 1901 in the context of floristic analysis of plant species distributions in the Alps and Jura [1]. He proposed that similarity between two species assemblages should be measured by the ratio of their shared species to the total number of unique species. This formulation has since become a canonical measure of set similarity, with applications extending to biology, computer science, and ontology engineering.

16.3 Conceptual Foundations

Given two finite sets S_i and S_j, the **Jaccard index** $J(S_i, S_j)$ is defined as:

$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}, \quad 0 \leq J(S_i, S_j) \leq 1.$$

The corresponding **Jaccard distance** is:

$$d_J(S_i, S_j) = 1 - J(S_i, S_j),$$

which satisfies the axioms of a metric [6].

Two distinct levels of application exist in ontology analysis:

1. **Element-level Jaccard:** evaluates similarity between *elements* (e.g., atomic units such as features or attributes) based on the sets in which they appear. Useful for identifying elements that belong to multiple categories.
2. **Set-level Jaccard:** evaluates similarity between *sets* (e.g., categories or clusters) based on their element memberships. Decisive for assessing whether categories remain distinct.

16.4 Abstract Algorithm

Let $\mathcal{S} = \{S_1, S_2, \ldots, S_n\}$ be the collection of sets in an ontology.

1. For each pair (S_i, S_j), compute:
$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$$
2. Construct the Jaccard similarity matrix $J = [J(S_i, S_j)]_{i,j=1}^n$.
3. Visualize J as a heatmap, where darker or redder regions denote higher similarity.

16.5 Python Implementation

A simple Python algorithm for constructing the Jaccard heatmap is:

```
import numpy as np
import matplotlib.pyplot as plt

def jaccard_index(setA, setB):
    return len(setA & setB) / len(setA | setB) if setA | setB else 0

def jaccard_heatmap(ontology_sets):
    n = len(ontology_sets)
    J = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            J[i, j] = jaccard_index(ontology_sets[i], ontology_sets[j])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity")
    plt.title("Jaccard Heatmap")
    plt.show()
    return J
```

16.6 Interpretation of Values

- $J = 0$: sets are disjoint.
- $0 < J < 0.3$: weak overlap, generally acceptable.
- $0.3 \leq J < 0.6$: moderate overlap, suggesting redundancy.
- $0.6 \leq J < 1$: strong overlap, indicating potential collapse of categorical boundaries.
- $J = 1$: complete overlap, sets are identical.

16.7 Significance in Ontological Analysis

The Jaccard heatmap provides:

1. A quantitative diagnostic for redundancy in ontologies.
2. A visual tool for detecting indistinct categorical boundaries.

3. A foundation for integrating similarity measures with topological analysis [3] and dimensionality reduction [4].

16.8 Conclusion

The Jaccard index, first developed for ecological analysis, is now central to computational ontology evaluation. At the element level it identifies cross-category elements; at the set level it assesses category distinctiveness. The heatmap visualization of pairwise Jaccard similarities thus provides a rigorous and interpretable diagnostic for assessing the structural validity of an ontology.

References

[1] Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37(142), 547–579.

[1] Pearson, K. (1895). Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an Ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies* (pp. 1–17). Springer.

[3] Brank, J., Grobelnik, M., & Mladenic, D. (2005). A survey of ontology evaluation techniques. In *Proceedings of the 8th International Multi-Conference on Information Society* (pp. 166–169).

[5] Rayson, P., & Garside, R. (2000). Comparing corpora using frequency profiling. *Literary and Linguistic Computing*, 16(3), 251–265.

[6] Kosub, S. (2019). A note on the triangle inequality for the Jaccard distance. *arXiv preprint arXiv:1612.02696*.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.

Chapter 17
Jaccard Element-Level Analysis

Abstract

This chapter presents the use of the Jaccard index at the element level. Here, elements are represented by the sets they belong to, and similarity is defined by overlap in set-membership profiles. This provides a diagnostic for promiscuous or cross-cutting elements that appear across multiple categories.

17.1 Mathematical formalism

Let $\mathcal{S} = \{S_1, \ldots, S_n\}$ be the ontology sets, with universe U of elements. For each element $e \in U$, define its profile:

$$P(e) = \{i \mid e \in S_i\}.$$

For two elements $e, f \in U$, the Jaccard index is:

$$J_{\text{elem}}(e, f) = \frac{|P(e) \cap P(f)|}{|P(e) \cup P(f)|}.$$

17.2 Pseudocode

```
\Procedure{ComputeElementLevelJaccard}{Sets S[1..n]}
  U = union of all elements
  for each element e in U:
    P[e] = { i | e     S[i] }
  for each pair (e,f):
    inter = |P[e]     P[f]|
    union = |P[e]     P[f]|
    J_elem[e,f] = inter / union (0 if union=0)
  return J_elem
\EndProcedure
```

17.3 Python implementation

```
import os, numpy as np, matplotlib.pyplot as plt

def jaccard_index(setA, setB):
    denom = len(setA | setB)
    return (len(setA & setB) / denom) if denom > 0 else 0.0

def jaccard_heatmap_elements(ontology_sets, out_dir="."):
    all_elements = sorted(set().union(*ontology_sets))
    profiles = {e: set() for e in all_elements}
    for si, s in enumerate(ontology_sets):
        for e in s: profiles[e].add(si)
    m = len(all_elements)
    J = np.zeros((m, m))
    for i,e1 in enumerate(all_elements):
        for j,e2 in enumerate(all_elements):
            J[i,j] = jaccard_index(profiles[e1], profiles[e2])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity (Elements)")
    plt.title("Jaccard Heatmap (Element-Level)")
    out_path = os.path.join(out_dir, "jacquard_elements.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return J, all_elements, out_path
```

17.4 Interpretation

- High $J_{\text{elem}}(e, f)$ means elements co-occur in many of the same categories.
- Elements with broadly overlapping profiles are candidates for review or redefinition.
- This analysis highlights the sources of redundancy in ontologies.

Chapter 18
Jaccard Element-Level Analysis

Abstract

This chapter presents the use of the Jaccard index at the element level. Here, elements are represented by the sets they belong to, and similarity is defined by overlap in set-membership profiles. This provides a diagnostic for promiscuous or cross-cutting elements that appear across multiple categories.

18.1 Mathematical formalism

Let $\mathcal{S} = \{S_1, \ldots, S_n\}$ be the ontology sets, with universe U of elements. For each element $e \in U$, define its profile:

$$P(e) = \{i \mid e \in S_i\}.$$

For two elements $e, f \in U$, the Jaccard index is:

$$J_{\text{elem}}(e, f) = \frac{|P(e) \cap P(f)|}{|P(e) \cup P(f)|}.$$

18.2 Pseudocode

```
\Procedure{ComputeElementLevelJaccard}{Sets S[1..n]}
  U = union of all elements
  for each element e in U:
    P[e] = { i | e     S[i] }
\chapter{Jaccard Set-Level Analysis}

\begin{abstract}
This chapter develops the Jaccard index at the set level, where categories are
    compared directly
through their constituent elements. This provides a diagnostic of category
    distinctiveness
and redundancy across an ontology.
\end{abstract}

\section{Mathematical formalism}
For two sets $S_i,S_j$:
\[
J_{\text{set}}(S_i,S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}.
\]
In incidence-matrix terms:
\[
J_{\text{set}}(S_i,S_j) = \frac{c_i^\top c_j}{\|c_i\|_1 + \|c_j\|_1 - c_i^\top c_j},
\]
where $c_i$ is the incidence column of $S_i$.

\section{Pseudocode}
\begin{pseudocode}
\Procedure{ComputeSetLevelJaccard}{Sets S[1..n]}
  for i = 1..n:
    for j = i..n:
      inter = |S[i]     S[j]|
      union = |S[i]     S[j]|
      J[i,j] = inter / union (0 if union=0)
      J[j,i] = J[i,j]
  return J
\EndProcedure
```

18.3 Python implementation

```
def jaccard_heatmap_sets(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    J = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            J[i,j] = jaccard_index(ontology_sets[i], ontology_sets[j])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity (Sets)")
    plt.title("Jaccard Heatmap (Set-Level)")
    out_path = os.path.join(out_dir, "jacquard_sets.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return J, out_path
```

18.4 Interpretation

- $J_{\text{set}}(S_i, S_j) = 0$: sets are disjoint (orthogonal categories).

- $0.3 \leq J < 0.6$: moderate redundancy, blurred boundaries.
- $J \geq 0.6$: strong overlap, near indistinguishability.

18.5 Conclusion

Set-level Jaccard quantifies category similarity directly from shared elements, providing a rigorous diagnostic of ontology structure.

Chapter 19
Relating Element-Level and Set-Level Jaccard Analysis

Abstract

This chapter compares Jaccard similarity computed at the element level and the set level. At the element level, Jaccard quantifies the similarity between two elements in terms of shared category memberships. At the set level, Jaccard quantifies similarity between categories in terms of shared elements. Although both measures are derived from the same incidence matrix, they emphasize different aspects of redundancy and overlap. A resolution section provides guidance on interpreting divergences between the two projections [1, 1].

19.1 Conceptual foundations

The Jaccard index is a classical measure of similarity, defined as:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

In ontology analysis, it can be applied at two levels:

- **Element-level Jaccard:** similarity of elements based on their sets of category memberships.

$$J(e, f) = \frac{|P(e) \cap P(f)|}{|P(e) \cup P(f)|}.$$

- **Set-level Jaccard:** similarity of categories based on their constituent elements.

$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}.$$

Both produce similarity matrices, which can be visualized as heatmaps.

19.2 Mathematical relationship

Let M be the incidence matrix ($M_{ij} = 1$ if $e_i \in S_j$).

$$\begin{aligned} \text{Element projection:} \quad J(e, f) &= \frac{(MM^\top)_{ef}}{|P(e) \cup P(f)|}, \\ \text{Set projection:} \quad J(S_i, S_j) &= \frac{(M^\top M)_{ij}}{|S_i \cup S_j|}. \end{aligned}$$

While both are derived from M, the denominators differ (element unions vs. set unions), so alignment is not guaranteed.

19.3 Pseudocode

19.3.1 Element-level Jaccard

```
\Procedure{ElementJaccard}{Ontology sets S}
  Elements = union of all sets
  for each pair (e,f):
    A = set of categories containing e
    B = set of categories containing f
    J[e,f] = |A     B| / |A     B|
  return J
\EndProcedure
```

19.3.2 Set-level Jaccard

```
\Procedure{SetJaccard}{Ontology sets S}
  for each pair (Si,Sj):
    J[i,j] = |Si     Sj| / |Si     Sj|
  return J
\EndProcedure
```

19.4 Python implementation

```
import numpy as np
import matplotlib.pyplot as plt
import itertools, os

def jaccard_index(A,B):
    return len(A & B) / len(A | B) if A | B else 0

def element_jaccard_heatmap(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    n = len(elements)
    J = np.zeros((n,n))
    memberships = {e: {i for i,S in enumerate(ontology_sets) if e in S} for e in
        elements}
    for i, e1 in enumerate(elements):
        for j, e2 in enumerate(elements):
            J[i,j] = jaccard_index(memberships[e1], memberships[e2])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity")
    plt.title("Element-Level Jaccard Heatmap")
    out_path = os.path.join(out_dir,"jaccard_elements.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return J, out_path

def set_jaccard_heatmap(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    J = np.zeros((n,n))
    for i, Si in enumerate(ontology_sets):
        for j, Sj in enumerate(ontology_sets):
            J[i,j] = jaccard_index(Si, Sj)
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity")
    plt.title("Set-Level Jaccard Heatmap")
    out_path = os.path.join(out_dir,"jaccard_sets.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return J, out_path
```

19.5 Analysis

- Element-level Jaccard captures the degree to which elements behave as "semantic twins," sharing similar category memberships.
- Set-level Jaccard captures category similarity — the extent to which categories collapse into one another.
- Comparing both can reveal whether redundancy is located in the categories or in the elements.

19.6 Resolution: Interpreting Differences

- **High Jaccard at element level, low at set level:** Elements are semantically overloaded, but categories remain distinct.
- **High Jaccard at set level, low at element level:** Categories collapse into each other, while elements remain specific.
- **Both high:** Total redundancy: neither categories nor elements are distinct.
- **Both low:** Well-structured ontology: distinct categories and elements.

References

[1] Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et du Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies* (pp. 1–17). Springer.

Chapter 20
Element-Level Jaccard Analysis vs Set-Level Jaccard Analysis

20.1 Overview

This section gives a formal account of two complementary uses of the Jaccard index in ontology analysis:

- **Set-level Jaccard:** direct comparison of category sets (categories as units).
- **Element-level Jaccard:** comparison of elements by their set-membership profiles (elements as units).

Both computations use the same binary set-similarity operator (Jaccard), but they are applied to different universes (sets vs. set-index vectors), which yields distinct diagnostics and interpretations [1, 1].

20.2 Mathematical formalism

Let the ontology be represented as a collection of n sets (categories)

$$\mathcal{S} = \{S_1, S_2, \ldots, S_n\}, \qquad S_i \subseteq U,$$

where U denotes the universe of atomic *elements* (features, items, symptoms, attributes, etc.). Let $m = |U|$ be the number of distinct elements.

Define the **indicator / incidence matrix** $M \in \{0,1\}^{m \times n}$ with entries

$$M_{e,i} = \begin{cases} 1 & \text{if element } e \in S_i, \\ 0 & \text{otherwise,} \end{cases} \qquad e \in U,\ i \in \{1, \ldots, n\}.$$

20.3 Set-level Jaccard

For two sets $S_i, S_j \in \mathcal{S}$ the Jaccard index is

$$J_{\text{set}}(S_i, S_j) \;=\; \frac{|S_i \cap S_j|}{|S_i \cup S_j|} \;=\; \frac{\sum_{e \in U} \mathbf{1}\{e \in S_i \wedge e \in S_j\}}{\sum_{e \in U} \mathbf{1}\{e \in S_i \vee e \in S_j\}}.$$

In incidence-matrix terms, let column vectors c_i and c_j be the i-th and j-th columns of M. Then

$$|S_i \cap S_j| = c_i^\top c_j, \qquad |S_i \cup S_j| = \|c_i\|_1 + \|c_j\|_1 - c_i^\top c_j,$$

so

$$J_{\text{set}}(S_i, S_j) = \frac{c_i^\top c_j}{\|c_i\|_1 + \|c_j\|_1 - c_i^\top c_j}.$$

20.4 Element-level Jaccard

Form a profile for each element $e \in U$ as the set of indices of categories containing e:

$$P(e) \;=\; \{\, i \in \{1, \ldots, n\} : M_{e,i} = 1 \,\}.$$

This is equivalently the e-th row of M viewed as a set of set-indices. For two elements $e, f \in U$:

$$J_{\text{elem}}(e, f) \;=\; \frac{|P(e) \cap P(f)|}{|P(e) \cup P(f)|} \;=\; \frac{\sum_{i=1}^{n} M_{e,i} M_{f,i}}{\sum_{i=1}^{n} \max(M_{e,i}, M_{f,i})}.$$

In row-vector notation, if r_e and r_f are rows of M,

$$J_{\text{elem}}(e, f) = \frac{r_e r_f^\top}{\|r_e\|_1 + \|r_f\|_1 - r_e r_f^\top}.$$

20.5 Key conceptual distinction

- **Set-level Jaccard** answers: "How similar are two categories in terms of their elements?" — this diagnoses category redundancy and boundary collapse.
- **Element-level Jaccard** answers: "How similar are two elements in terms of the categories they belong to?" — this identifies promiscuous elements that join multiple categories and may be the source of category entanglement.

20.6 Pseudocode (abstract)

```
\Procedure{ComputeSetLevelJaccard}{Sets S[1..n]}
  for i = 1 to n:
    for j = i to n:
      inter = | S[i]     S[j] |
      union = | S[i]     S[j] |
      if union == 0:
         J[i,j] = 0
      else:
         J[i,j] = inter / union
      J[j,i] = J[i,j]
  return J
\EndProcedure
```

```
\Procedure{ComputeElementLevelJaccard}{Sets S[1..n]}
  U = union of all elements across S[1..n]
  for each element e in U:
    P[e] = { i | e     S[i] }   // element's profile: set of category indices
  let m = |U|
  for a = 1 to m:
    for b = a to m:
      inter = | P[a]     P[b] |
      union = | P[a]     P[b] |
      if union == 0:
        J_elem[a,b] = 0
      else:
        J_elem[a,b] = inter / union
      J_elem[b,a] = J_elem[a,b]
  return J_elem
\EndProcedure
```

20.7 Python implementations

```python
import os
import numpy as np
import matplotlib.pyplot as plt

def jaccard_index(setA, setB):
    """Safe Jaccard; returns 0 if both sets are empty."""
    denom = len(setA | setB)
    return (len(setA & setB) / denom) if denom > 0 else 0.0

def jaccard_heatmap_sets(ontology_sets, out_dir="."):
    """
    Set-level Jaccard heatmap.
    - ontology_sets: list of Python sets (each set = category's elements)
    - out_dir: directory prefix where the fixed filename 'jacquard_sets.png' will be
        written
    Returns: (J_matrix, out_path)
    """
    n = len(ontology_sets)
    J = np.zeros((n, n), dtype=float)
    for i in range(n):
        for j in range(n):
            J[i, j] = jaccard_index(ontology_sets[i], ontology_sets[j])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity (sets)")
    plt.title("Jaccard Heatmap (Set-Level)")
    out_path = os.path.join(out_dir, "jacquard_sets.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return J, out_path

def jaccard_heatmap_elements(ontology_sets, out_dir="."):
    """
    Element-level Jaccard heatmap.
    - ontology_sets: list of Python sets
    - out_dir: directory prefix where the fixed filename 'jacquard_elements.png' will
         be written
    Returns: (J_matrix, element_list, out_path)
    """
    # collect all elements
    all_elements = sorted(set().union(*ontology_sets)) if ontology_sets else []
    # build element -> profile (set indices)
    element_profiles = {e: set() for e in all_elements}
    for si, s in enumerate(ontology_sets):
        for e in s:
            element_profiles[e].add(si)
    m = len(all_elements)
    J = np.zeros((m, m), dtype=float)
    for a, ea in enumerate(all_elements):
        for b, eb in enumerate(all_elements):
            J[a, b] = jaccard_index(element_profiles[ea], element_profiles[eb])
    plt.imshow(J, cmap="hot", interpolation="nearest")
    plt.colorbar(label="Jaccard Similarity (elements)")
    plt.title("Jaccard Heatmap (Element-Level)")
    out_path = os.path.join(out_dir, "jacquard_elements.png")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return J, all_elements, out_path
```

20.8 Complexity and implementation notes

- Let n be number of sets and m the number of distinct elements.

  - Set-level pairwise computation requires $O(n^2 \cdot \alpha)$ set-operations, where α is cost of computing intersection/union on average (depends on set sizes). If average set size is s, complexity is $O(n^2 s)$.
  - Element-level pairwise computation requires $O(m^2 \cdot \beta)$ operations where β depends on average profile size (number of sets an element belongs to). In sparse ontologies m can be much larger than n, so element-level may be more expensive.
- Memory: storing full element-level matrix costs $O(m^2)$, which may be infeasible for very large vocabularies; consider thresholded or sampled analysis for scale.
- Edge cases:
  - If both sets (or profiles) are empty, define $J = 0$ by convention.
  - If many singleton elements or singleton sets exist, raw Jaccard may overstate sparsity — consider smoothing or weighted Jaccard variants.

20.9 Interpretation guidance

- Use **set-level Jaccard** to assess category distinctiveness: large contiguous high-similarity blocks indicate groups of categories that largely share elements (potential collapse).
- Use **element-level Jaccard** to find *shared features*: elements with broad, similar profiles are candidates for redefinition, conditioning, or removal if they undermine category separation.
- Combine the two views: high set-level Jaccard often correlates with a population of elements having high element-level centrality or promiscuity; the two diagnostics together help locate causal elements.

20.10 Practical recommendations

- Compute both matrices for a representative sample before running a full analysis at scale.
- Use sparse matrix representations for large m or n.
- Consider augmenting Jaccard with TF–IDF weighting or mutual-information scores if elements have differing diagnostic importance.

Volume III

Topology and Statistics

Part VI
Betti Curves

Chapter 21
Persistent Homology and Betti Curves: Foundations

Abstract

This chapter introduces persistent homology and Betti curves as a method for analyzing the topology of complex data. Betti numbers quantify connected components, loops, and higher-dimensional holes in a simplicial complex, and their evolution across a filtration yields Betti curves. These tools provide a robust framework for identifying structural redundancy and connectivity patterns in ontological graphs [3, 1].

21.1 Historical perspective

Topological data analysis (TDA) emerged in the late 20th century, drawing from algebraic topology [3] and computational geometry [1]. Persistent homology, introduced by Edelsbrunner and colleagues [1, 4], tracks the birth and death of topological features across scales. Betti curves summarize this persistence by plotting the number of features of each dimension across thresholds. This has been widely applied in biology, neuroscience, and complex networks [7, 2].

21.2 Mathematical formalism

Let $G = (V, E)$ be a weighted graph, where weights represent set intersections. For threshold θ, define the subgraph:

$$G_\theta = (V, \{e \in E \mid w(e) \geq \theta\}).$$

Construct the clique complex K_θ from G_θ. The k-th Betti number is:

$$\beta_k(\theta) = \dim H_k(K_\theta),$$

where H_k is the k-th homology group. The Betti curve is β_k plotted as a function of θ.

21.3 Pseudocode

```
\Procedure{ComputeBettiCurves}{Graph G(V,E,w)}
  thresholds = sorted unique edge weights
  for    in thresholds:
    G   = subgraph with edges w(e)
    K   = clique complex of  G
    for k = 0..d:
       _k [  ] = rank of homology group H_k( K  )
  return { _k  curves}
\EndProcedure
```

21.4 Python implementation

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from gudhi import SimplexTree

def betti_curves(graph, out_dir="."):
    weights = sorted({d['weight'] for u,v,d in graph.edges(data=True)}, reverse=True)
    betti = {0: [], 1: []}
    for th in weights:
        Gt = nx.Graph(((u,v,d) for u,v,d in graph.edges(data=True) if d['weight']>=th
            ))
        st = SimplexTree()
        for clique in nx.find_cliques(Gt):
            for k in range(1,len(clique)+1):
                for simplex in itertools.combinations(clique,k):
                    st.insert(simplex)
        st.compute_persistence()
        betti[0].append(st.betti_numbers()[0])
        betti[1].append(st.betti_numbers()[1])
    for k in betti:
        plt.plot(weights, betti[k], label=f" _ {k}")
    plt.xlabel("Threshold")
    plt.ylabel("Betti Number")
    plt.legend()
    out_path = os.path.join(out_dir,"betti_curves.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return betti, out_path
```

21.5 Analysis

Betti curves reveal how topological features persist across scales. β_0 tracks connected components, while β_1 tracks loops. Sharp changes indicate thresholds where connectivity or cycles emerge/disappear.

21.6 Interpretation

- Flat β_0: indicates cohesive structure across thresholds.
- Rising β_1: cycles in the ontology structure, often due to redundant overlaps.
- Long-persistent features: robust structural motifs of the ontology.

References

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction.* American Mathematical Society.

[1] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

[3] Hatcher, A. (2002). *Algebraic Topology.* Cambridge University Press.

[1] Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2000). Topological persistence and simplification. In *Proceedings 41st Annual Symposium on Foundations of Computer Science* (pp. 454–463). IEEE.

[4] Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274.

[2] Ghrist, R. (2008). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

[7] Sizemore, A. E., Giusti, C., & Bassett, D. S. (2018). Classification of weighted networks through mesoscale homological features. *Journal of Complex Networks*, 5(2), 245–273.

Chapter 22
Betti Curves on Ontology Overlap Graph (Element-Level)

Abstract

This chapter applies persistent homology and Betti curves to the element-level projection of an ontology. Elements are treated as nodes in a graph, and edges are weighted by the number of categories (sets) that the two elements share. Thresholding this graph by edge weight yields a filtration, from which Betti curves are derived. This analysis reveals structural motifs among elements, highlighting redundancy and multi-category entanglement [7, 2].

22.1 Historical perspective

Element co-occurrence has been a foundational technique in statistical and semantic analysis [1, 2, 3]. The integration of persistent homology into co-occurrence networks extends this tradition by adding topological diagnostics, allowing one to detect loops, clusters, and higher-order patterns that go beyond pairwise overlaps [1, 4]. In ontology analysis, this approach provides a robust diagnostic of redundancy in the element space.

22.2 Mathematical formalism

Let U be the universe of elements, and let $P(e) = \{i \mid e \in S_i\}$ denote the set of categories containing element e. Define the element overlap graph $G = (U, E)$, with edge weights:

$$w(e, f) = |P(e) \cap P(f)|.$$

For threshold θ, define the subgraph:

$$G_\theta = (U, \{(e, f) \mid w(e, f) \geq \theta\}).$$

Construct the clique complex K_θ from G_θ. Betti numbers are defined as:

$$\beta_k(\theta) = \dim H_k(K_\theta),$$

with β_0 tracking connected components and β_1 loops among elements. The Betti curve is β_k plotted as a function of θ.

22.3 Pseudocode

```
\Procedure{ComputeElementBettiCurves}{Ontology sets S[1..n]}
  Build element graph G:
    Nodes = all elements
    Edge weight w(e,f) = |{i | e,f     S[i]}|
  thresholds = sorted unique edge weights
  for    in thresholds:
    G   = subgraph with edges w(e,f)
    K   = clique complex of G
    for k = 0..d:
       _k [  ] = rank of homology group H_k( K  )
  return { _k  curves}
\EndProcedure
```

22.4 Python implementation

```python
import itertools, os
import networkx as nx
import matplotlib.pyplot as plt
from gudhi import SimplexTree

def betti_curves_elements(ontology_sets, out_dir="."):
    # Build element overlap graph
    elements = sorted(set().union(*ontology_sets))
    G = nx.Graph()
    G.add_nodes_from(elements)
    for e1, e2 in itertools.combinations(elements, 2):
        weight = sum(1 for s in ontology_sets if e1 in s and e2 in s)
        if weight > 0:
            G.add_edge(e1, e2, weight=weight)

    # Threshold filtration
    weights = sorted({d['weight'] for _,_,d in G.edges(data=True)}, reverse=True)
    betti = {0: [], 1: []}
    for th in weights:
        Gt = nx.Graph(((u,v,d) for u,v,d in G.edges(data=True) if d['weight']>=th))
        st = SimplexTree()
        for clique in nx.find_cliques(Gt):
            for k in range(1, len(clique)+1):
                for simplex in itertools.combinations(clique, k):
                    st.insert(simplex)
        st.compute_persistence()
        bnums = st.betti_numbers()
        betti[0].append(bnums[0] if len(bnums)>0 else 0)
        betti[1].append(bnums[1] if len(bnums)>1 else 0)

    # Plot
    for k in betti:
        plt.plot(weights, betti[k], label=f" _ {k}")
    plt.xlabel("Threshold (Shared Categories)")
    plt.ylabel("Betti Number")
    plt.legend()
    plt.title("Element-Level Betti Curves")
    out_path = os.path.join(out_dir,"betti_elements.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return betti, out_path
```

22.5 Analysis

Element-level Betti curves track the persistence of topological features among elements. β_0 decreases as elements form connected clusters at lower thresholds. β_1 rises when cycles appear — indicating redundant triangular or higher-order entanglements between elements. The curve shape thus reflects redundancy, modularity, and overlap among elements.

22.6 Interpretation

- Flat β_0: elements consistently form a cohesive cluster.
- High β_1: many loops, suggesting redundancy in element assignments.
- Persistent cycles: robust element communities spanning multiple categories.

References

[1] Pearson, K. (1895). Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242.

[2] Lund, K., & Burgess, C. (1996). Producing high-dimensional semantic spaces from lexical co-occurrence. *Behavior Research Methods, Instruments, & Computers*, 28, 203–208.

[3] Landauer, T. K., & Dumais, S. T. (1997). A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge. *Psychological Review*, 104(2), 211–240.

[1] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

[4] Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274.

[2] Ghrist, R. (2008). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

[7] Sizemore, A. E., Giusti, C., & Bassett, D. S. (2018). Classification of weighted networks through mesoscale homological features. *Journal of Complex Networks*, 5(2), 245–273.

Chapter 23
Betti Curves on Ontology Overlap Graph (Set-Level)

Abstract

This chapter applies persistent homology and Betti curves to the set-level projection of an ontology. Categories (sets) are treated as nodes in a graph, with edges weighted by the number of shared elements between them. By thresholding this overlap graph and constructing clique complexes, we obtain Betti curves that track the evolution of connected components and cycles across scales. This provides a topological diagnostic of redundancy and collapse among categories [1, 2].

23.1 Historical perspective

Analyzing category overlap through graphs dates back to early classification and clustering research, where similarity matrices and overlap coefficients were central [1]. Persistent homology extends this tradition, by embedding the overlap graph into a filtration and quantifying higher-order structures with Betti numbers [3, 4]. In ontologies, this captures the degree to which categories are redundant, tightly coupled, or structurally entangled.

23.2 Mathematical formalism

Let $\mathcal{S} = \{S_1, \ldots, S_n\}$ be the ontology sets. Define the set overlap graph $G = (\mathcal{S}, E)$, with edge weights:

$$w(S_i, S_j) = |S_i \cap S_j|.$$

For threshold θ, define:

$$G_\theta = (\mathcal{S}, \{(S_i, S_j) \mid w(S_i, S_j) \geq \theta\}).$$

From G_θ, construct the clique complex K_θ. Betti numbers are defined as:

$$\beta_k(\theta) = \dim H_k(K_\theta),$$

with β_0 tracking connected components (disjoint groups of categories) and β_1 cycles (redundant overlap loops among categories).

23.3 Pseudocode

```
\Procedure{ComputeSetBettiCurves}{Ontology sets S[1..n]}
  Build set graph G:
    Nodes = S[i]
    Edge weight w(i,j) = |S[i]     S[j]|
  thresholds = sorted unique edge weights
  for    in thresholds:
    G   = subgraph with edges w(i,j)
    K   = clique complex of G
    for k = 0..d:
       _k [  ] = rank of homology group H_k( K  )
  return { _k  curves}
\EndProcedure
```

23.4 Python implementation

```
import itertools, os
import networkx as nx
import matplotlib.pyplot as plt
from gudhi import SimplexTree

def betti_curves_sets(ontology_sets, out_dir="."):
    n = len(ontology_sets)
    G = nx.Graph()
    G.add_nodes_from(range(n))
    for i, j in itertools.combinations(range(n), 2):
        weight = len(ontology_sets[i] & ontology_sets[j])
        if weight > 0:
            G.add_edge(i, j, weight=weight)

    weights = sorted({d['weight'] for _,_,d in G.edges(data=True)}, reverse=True)
    betti = {0: [], 1: []}
    for th in weights:
        Gt = nx.Graph(((u,v,d) for u,v,d in G.edges(data=True) if d['weight']>=th))
        st = SimplexTree()
        for clique in nx.find_cliques(Gt):
            for k in range(1, len(clique)+1):
                for simplex in itertools.combinations(clique, k):
                    st.insert(simplex)
        st.compute_persistence()
        bnums = st.betti_numbers()
        betti[0].append(bnums[0] if len(bnums)>0 else 0)
        betti[1].append(bnums[1] if len(bnums)>1 else 0)

    for k in betti:
        plt.plot(weights, betti[k], label=f" _ {k}")
    plt.xlabel("Threshold (Shared Elements)")
    plt.ylabel("Betti Number")
    plt.legend()
    plt.title("Set-Level Betti Curves")
    out_path = os.path.join(out_dir,"betti_sets.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return betti, out_path
```

23.5 Analysis

Set-level Betti curves reveal how categories merge, fragment, and form loops as the overlap threshold changes. β_0 decreases as categories merge into connected groups with high overlap. β_1 increases when redundant triangular overlaps or cycles occur, indicating structural collapse of categorical distinctions.

23.6 Interpretation

- High β_0 at large thresholds: categories are distinct.
- Rapidly falling β_0: categories quickly merge, showing weak distinctiveness.
- Rising β_1: redundancy loops, suggesting over-connected categories.
- Persistent cycles: entrenched structural redundancies in the ontology.

References

[1] Sørensen, T. (1948). A method of establishing groups of equal amplitude in plant sociology based on similarity of species and its application to analyses of the vegetation on Danish commons. *Biologiske Skrifter*, 5(4), 1–34.

[1] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction.* American Mathematical Society.

[4] Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274.

[2] Ghrist, R. (2008). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

Chapter 24
Relating Set-Level and Element-Level Betti Curves

Abstract

This chapter examines the relationship between Betti curves derived from the set-level and element-level projections of an ontology. Although both originate from the same underlying incidence structure, they emphasize different aspects: categories versus their constituent elements. By comparing the two, we can resolve structural ambiguities, and understand how redundancy in one projection manifests in the other [1, 2].

24.1 Conceptual foundations

An ontology can be represented by an incidence matrix M, where $M_{ij} = 1$ if element e_i belongs to set S_j. Two natural projections exist:

- **Element-level graph:** nodes are elements, edges weighted by co-membership in sets (rows of $MM^\top$).
- **Set-level graph:** nodes are sets, edges weighted by element overlap (columns of $M^\top M$).

Betti curves computed on these two graphs quantify the evolution of topological features under threshold filtrations. Although related by duality, their Betti signatures often diverge.

24.2 Mathematical relationship

Let $A = MM^\top$ (element co-occurrence matrix) and $B = M^\top M$ (set overlap matrix). Then:

$$A_{ef} = |P(e) \cap P(f)|, \quad B_{ij} = |S_i \cap S_j|.$$

Both encode redundancy, but from different perspectives:

- $\beta_k(A)$ reflects entanglement among elements across categories.
- $\beta_k(B)$ reflects entanglement among categories via shared elements.

A divergence in their Betti curves implies asymmetry between how categories are organized versus how elements are distributed.

24.3 Pseudocode for comparative analysis

```
\Procedure{CompareElementSetBettiCurves}{Ontology}
  A = M M^T   # element projection
  B = M^T M   # set projection
    _elem   = ComputeBettiCurves(A)
    _set    = ComputeBettiCurves(B)
  Plot( _elem , _set )
  return Differences( _elem , _set )
\EndProcedure
```

24.4 Python illustration

```
def compare_betti(element_betti, set_betti, weights, out_dir="."):
    import matplotlib.pyplot as plt
    plt.plot(weights, element_betti[0], 'b-', label=" 0  (Elements)")
    plt.plot(weights, set_betti[0], 'r--', label=" 0  (Sets)")
    plt.plot(weights, element_betti[1], 'g-', label=" 1  (Elements)")
    plt.plot(weights, set_betti[1], 'm--', label=" 1  (Sets)")
    plt.xlabel("Threshold")
    plt.ylabel("Betti Number")
    plt.legend()
    plt.title("Element vs. Set Betti Curves")
    out_path = os.path.join(out_dir,"betti_comparison.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return out_path
```

24.5 Analysis

- When element and set Betti curves **align**, redundancy is symmetric: overlapping categories mirror entangled elements.
- When β_0 differs: categories merge faster than elements, or vice versa — suggesting imbalance in how overlap is distributed.
- When β_1 differs: cycles appear at one level but not the other, indicating redundancy among categories without equivalent redundancy among elements (or the reverse).

24.6 Resolution

Divergences between set-level and element-level Betti curves highlight structural inconsistencies in the ontology:

- **High β_1 at set level, low at element level:** categories are redundant, but elements remain relatively distinct. This indicates collapse of taxonomic boundaries.
- **High β_1 at element level, low at set level:** elements are multiply entangled, but categories retain coherence. This indicates semantic overloading of elements.
- **Asymmetric β_0:** rapid merging on one projection but not the other signals imbalance in granularity versus redundancy.

Thus, comparing the two projections provides a diagnostic of whether redundancy lies in the categorization scheme or in the elemental assignments.

References

[1] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

[2] Ghrist, R. (2008). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.

[4] Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274.

Part VII
Persistent Homology

Chapter 25
Persistent Homology (General)

Abstract

Persistent Homology (PH) is a method from Topological Data Analysis (TDA) that tracks the emergence and disappearance of topological features (such as connected components, loops, and voids) across scales of analysis. In ontology evaluation, PH provides a way to detect structural stability or fragility by examining overlap graphs under increasing thresholds. This chapter introduces PH, its conceptual foundations, and its application to ontology analysis.

25.1 Introduction

Ontologies can be represented as graphs of overlapping sets. Persistent Homology analyzes the evolution of these graphs under threshold filtration, revealing whether structural features persist across scales or vanish quickly, indicating instability.

25.2 Historical Perspective

Topology's concern with invariants of shape dates back to Euler's polyhedral formula (1758) and Betti's homology groups (1871). The modern framework of persistent homology was developed by Edelsbrunner and colleagues in the 1990s–2000s [1, 3], providing a robust method for multi-scale analysis. PH has since been applied in computational biology, neuroscience, and data mining. In ontology analysis, PH carries this legacy into conceptual structures.

25.3 Conceptual Foundations

- Input: Ontology sets $\rightarrow$ build overlap graph. - Threshold filtration: Add edges if intersection size $\geq t$. - Track Betti numbers $\beta_0, \beta_1, \beta_2, ...$ as t increases.

25.4 Mathematical Formalism

Let G_t be the graph at threshold t. Define Betti numbers as:

$$\beta_k(G_t) = \text{rank}(H_k(G_t))$$

where H_k is the k-th homology group.

25.5 Pseudocode

```
Input: Overlap graph
For threshold t in range:
    build G_t with edges of weight     t
    compute Betti numbers ( 0 ,  1 , ...)
Track persistence of features
Output persistence diagrams/curves
```

25.6 Python Algorithm

```python
import networkx as nx
from itertools import combinations

def persistent_homology_curves(ontology_sets, thresholds):
    U = list(range(len(ontology_sets)))
    results = {}
    for t in thresholds:
        G = nx.Graph()
        G.add_nodes_from(U)
        for i, j in combinations(U, 2):
            if len(ontology_sets[i] & ontology_sets[j]) >= t:
                G.add_edge(i, j)
        results[t] = {
            "components": nx.number_connected_components(G),
            "cycles": len(nx.cycle_basis(G))
        }
    return results
```

25.7 Analysis and Interpretation

- Persistent β_0: stable clusters. - Persistent β_1: stable cycles (redundant overlaps). - Rapid collapse: fragile structure.

25.8 Significance

PH reveals the robustness of taxonomic structure, complementing similarity and redundancy metrics.

25.9 Conclusion

Persistent Homology provides a deep, topological lens into ontology structure.

References

References

[1] Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2000). Topological persistence and simplification. In *Proceedings 41st Annual Symposium on Foundations of Computer Science* (pp. 454–463). IEEE.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction.* American Mathematical Society.

Chapter 26
Persistent Homology: Element-Level Analysis

Abstract

Element-level Persistent Homology tracks the participation of individual elements in topological features across thresholds. It reveals whether certain elements stabilize or destabilize ontology structure.

26.1 Introduction

This perspective emphasizes micro-level contributions, asking: which elements create persistent overlaps and which dissolve quickly?

26.2 Historical Perspective

In early graph theory, Euler's bridges problem highlighted the role of individual nodes in cycles [1]. PH extends this reasoning by quantifying persistence of element-driven topological features.

26.3 Conceptual Foundations

Each element's membership profile can be projected onto the filtration process. Elements repeatedly forming cycles or bridges across sets indicate redundancy.

26.4 Mathematical Formalism

For element e, define persistence as:

$$PH_{element}(e) = \sum_t f(e,t)$$

where $f(e,t)$ measures e's contribution to β_0 and β_1 at threshold t.

26.5 Pseudocode

```
For each element e:
   For each threshold t:
      build G_t
      if e contributes to cycle/component:
          record persistence
Aggregate persistence across thresholds
```

26.6 Python Algorithm

```python
def element_persistence(ontology_sets, thresholds):
    contributions = {e:0 for e in set().union(*ontology_sets)}
    for t in thresholds:
        results = persistent_homology_curves(ontology_sets, [t])
        # here: compute contributions by elements
        # placeholder: mark elements in overlaps >= t
        for S in ontology_sets:
            for e in S:
                contributions[e] += 1
    return contributions
```

26.7 Analysis

- High persistence: element anchors redundancy. - Low persistence: element minimally overlaps.

26.8 Significance

Element-level PH localizes collapse drivers in the ontology.

26.9 Conclusion

Tracking persistence of elements reveals their structural role in redundancy.

References

References

[1] Euler, L. (1736). Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, 8, 128–140.

Chapter 27
Persistent Homology: Set-Level Analysis

Abstract

Set-level Persistent Homology analyzes the topology of category overlaps, identifying whether sets cluster or cycle persistently across thresholds.

27.1 Introduction

This macro perspective considers sets as nodes, examining the evolution of their connectivity under filtration.

27.2 Historical Perspective

In sociology, Simmel emphasized the role of cliques and cycles in social networks [1]. PH extends this by providing formal persistence measures for such structures in ontologies.

27.3 Conceptual Foundations

At threshold t, connect sets with overlaps $\geq t$. Compute Betti numbers to track persistence of connectivity and cycles.

27.4 Mathematical Formalism

$$PH_{set}(t) = (\beta_0(G_t), \beta_1(G_t))$$

27.5 Pseudocode

```
For each threshold t:
   build overlap graph
   compute Betti numbers
Track curves of  0  and  1
```

27.6 Python Algorithm

```
def set_persistence_curves(ontology_sets, thresholds):
    return persistent_homology_curves(ontology_sets, thresholds)
```

27.7 Analysis

- Persistent β_0: stable partitions. - Persistent β_1: cycles showing redundancy.

27.8 Significance

Set-level PH reveals global structural stability.

27.9 Conclusion

Persistent homology curves show whether taxonomy resists or succumbs to collapse.

References

References

[1] Simmel, G. (1908). *Soziologie: Untersuchungen über die Formen der Vergesellschaftung.* Duncker & Humblot.

Chapter 28
Persistent Homology: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter compares element-level and set-level PH, analyzing divergences in persistence profiles and resolving interpretive tensions.

28.1 Introduction

Do element-driven cycles explain set-level collapse, or do sets collapse independently of specific elements? Comparing both levels clarifies the origin of redundancy.

28.2 Historical Perspective

The micro–macro tension recurs in many fields, from Durkheim's sociology to modern network science [1, 5]. Persistent Homology inherits this problem, requiring reconciliation of individual and structural persistence.

28.3 Conceptual Foundations

- Element-level PH: highlights overstretched terms. - Set-level PH: highlights systemic cycles.

28.4 Analysis

- Both high persistence: systemic redundancy. - Element high, set low: collapse driven by few elements. - Set high, element low: redundant clustering of sets.

28.5 Resolution

Differences guide whether refinement targets specific terms or systemic restructuring.

28.6 Conclusion

The interplay of element- and set-level PH provides a full account of ontological resilience.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.
[5] Newman, M. E. J. (2006). Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23), 8577–8582.

Volume IV

Statistics - Multivariate Analysis

Part VIII
Axis Independence Score

Chapter 29
Axis Independence Score (General)

Abstract

The Axis Independence Score (AIS) measures the statistical independence between different axes or clusters of an ontology. It indicates whether categories represent distinct conceptual dimensions or whether they collapse into correlated structures. This chapter introduces AIS, its mathematical basis, and its role in ontological diagnostics.

29.1 Introduction

Ontologies are often organized into dimensions or axes that are presumed independent. However, excessive correlation among axes undermines the structural integrity of the classification. AIS evaluates the degree of orthogonality between category incidence vectors, serving as a quantitative test of independence.

29.2 Historical Perspective

The idea of axis independence is rooted in classical geometry and logic, where orthogonality symbolized distinctness of categories. In statistics, Pearson [1] formalized correlation as a measure of dependence between variables. Later, factor analysis and PCA emphasized independence as a criterion for meaningful dimensionality reduction [4]. In ontology evaluation, AIS extends these traditions by applying correlation measures to categorical incidence data.

29.3 Conceptual Foundations

For two sets S_i and S_j, define binary incidence vectors across the universe U. Correlation between these vectors indicates the degree to which categories co-occur. AIS aggregates these correlations into a global measure of independence.

29.4 Mathematical Formalism

Let X_i and X_j be incidence vectors of sets S_i and S_j. Then:

$$AIS = 1 - \frac{2}{n(n-1)} \sum_{i<j} |\rho(X_i, X_j)|$$

where ρ is the Pearson correlation coefficient.

29.5 Pseudocode

```
Input: Sets S1, ..., Sn over universe U
For each set Si, build binary vector Xi over U
Compute correlations between all pairs (Xi, Xj)
AIS = 1 - average absolute correlation
Output AIS
```

29.6 Python Algorithm

```python
import numpy as np
from itertools import combinations

def axis_independence_score(ontology_sets):
    U = list(set().union(*ontology_sets))
    M = np.array([[1 if u in S else 0 for u in U] for S in ontology_sets])
    corrs = []
    for i, j in combinations(range(len(M)), 2):
        if np.std(M[i]) > 0 and np.std(M[j]) > 0:
            rho = np.corrcoef(M[i], M[j])[0, 1]
            corrs.append(abs(rho))
    return 1 - np.mean(corrs) if corrs else 1
```

29.7 Analysis and Interpretation

- $AIS \approx 1$: high independence, strong orthogonality. - $AIS \approx 0$: axes collapse into redundant structures. - Intermediate values: partial dependence.

29.8 Significance

AIS reveals whether the ontology truly spans independent conceptual dimensions or is artificially partitioned.

29.9 Conclusion

AIS is a powerful diagnostic of orthogonality, extending classical correlation analysis to ontological structures.

References

References

[1] Pearson, K. (1895). Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 30
Axis Independence Score: Element-Level Analysis

Abstract

Element-level AIS examines the degree to which individual elements contribute to correlations between axes. It reveals whether redundancy originates from a few overstretched elements or from systemic correlations.

30.1 Introduction

By tracking co-occurrence of elements across axes, element-level AIS identifies micro-drivers of dependence.

30.2 Historical Perspective

The notion of individual contributions to dependence echoes Fisher's work on analysis of variance [1]. In bibliometrics, Garfield noted that "bridging terms" linked otherwise independent clusters [2]. Element-level AIS adapts this tradition to ontologies.

30.3 Conceptual Foundations

For each element e, construct its incidence profile across sets. Correlation of these profiles across elements reveals redundancy drivers.

30.4 Mathematical Formalism

$$AIS_{element}(e) = 1 - \frac{2}{n(n-1)} \sum_{i<j} |\rho(X_i^e, X_j^e)|$$

30.5 Pseudocode

```
For each element e in U:
   Build vector Ve = [1 if e in Si else 0]
   For each pair (Vi, Vj):
       compute correlation
   AIS_element(e) = 1 - average absolute correlation
```

30.6 Python Algorithm

```python
def element_level_ais(ontology_sets):
    U = set().union(*ontology_sets)
    results = {}
    for e in U:
        vec = np.array([1 if e in S else 0 for S in ontology_sets])
        # correlation trivial if constant vector
        if np.std(vec) == 0:
            results[e] = 1.0
        else:
            results[e] = np.corrcoef(vec, vec)[0,0] # placeholder
    return results
```

30.7 Analysis

- Elements with low $AIS_{element}$: redundancy hubs. - Elements with high $AIS_{element}$: independent identifiers.

30.8 Significance

This identifies whether dependence is systemic or localized.

30.9 Conclusion

Element-level AIS localizes sources of ontological dependence.

References

References

[1] Fisher, R. A. (1925). *Statistical Methods for Research Workers*. Oliver and Boyd.
[2] Garfield, E. (1979). *Citation Indexing: Its Theory and Application*. Wiley.

Chapter 31
Axis Independence Score: Set-Level Analysis

Abstract

Set-level AIS evaluates correlations across sets considered as axes of classification. It measures whether the ontology's categories behave as distinct dimensions.

31.1 Introduction

Unlike element-level AIS, set-level AIS focuses on macro-structure, diagnosing collapse of axes due to correlated membership patterns.

31.2 Historical Perspective

The idea of dimensional independence has deep roots in psychometrics, where factor rotation was used to ensure independence of latent constructs [1]. In ontology design, independence across categories has long been a goal to avoid redundancy.

31.3 Conceptual Foundations

Binary incidence vectors of sets are compared for correlation. High correlations undermine orthogonality.

31.4 Mathematical Formalism

$$AIS_{set} = 1 - \frac{2}{n(n-1)} \sum_{i<j} |\rho(X_i, X_j)|$$

31.5 Pseudocode

```
For each pair of sets (Si, Sj):
    build incidence vectors Xi, Xj
    compute correlation
AIS_set = 1 - average absolute correlation
```

31.6 Python Algorithm

```
def set_level_ais(ontology_sets):
    return axis_independence_score(ontology_sets)
```

31.7 Analysis

- AIS_{set} close to 1: orthogonal axes. - AIS_{set} low: collapse into redundant categories.

31.8 Significance

Set-level AIS reveals whether categories contribute independent dimensions or duplicate one another.

31.9 Conclusion

Set-level AIS diagnoses the health of macro-structural independence.

References

References

[1] Thurstone, L. L. (1935). *The Vectors of Mind*. University of Chicago Press.

Chapter 32
Axis Independence Score: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter compares element-level and set-level AIS, highlighting divergences and their interpretive significance.

32.1 Introduction

Micro-level and macro-level measures of independence can yield different results. Understanding their interplay clarifies whether dependence originates from elements or categories.

32.2 Historical Perspective

This micro–macro tension mirrors debates in sociology, where Durkheim distinguished between individual and structural properties [1]. In ontology research, similar tensions arise between redundant terms and collapsed taxonomies.

32.3 Conceptual Foundations

- Element-level AIS: detects redundancy drivers. - Set-level AIS: detects structural collapse.

32.4 Analysis

- Both high: healthy ontology. - Element low, set high: redundancy localized in elements. - Set low, element high: structural duplication.

32.5 Resolution

Discrepancies suggest whether refinement is needed at the element or category level.

32.6 Conclusion

Interpreting both levels together provides a full picture of ontological independence.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.

Part IX
Principal Component Analysis

Chapter 33
Principal Component Analysis: Foundations

Abstract

This chapter introduces Principal Component Analysis (PCA) as a tool for dimensionality reduction in ontology analysis. PCA identifies latent axes of variation in the incidence matrix, revealing how much of the structure can be explained by a small number of orthogonal components. This chapter provides the historical foundations, mathematical formalism, pseudocode, and an implementation in Python [1, 4].

33.1 Historical perspective

PCA was introduced by Karl Pearson [1] as a technique for reducing multivariate data into orthogonal axes, later generalized by Hotelling (1933) and widely applied across sciences. Jolliffe's work [4] formalized modern PCA applications. In ontology analysis, PCA is used to test whether categories (or elements) collapse onto fewer latent dimensions.

33.2 Mathematical formalism

Given incidence matrix $M \in \{0,1\}^{m\times n}$:

$$X = M - \mu, \quad C = \frac{1}{n-1} X^\top X,$$

where C is the covariance matrix. Eigen-decomposition yields:

$$Cv_i = \lambda_i v_i, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq 0.$$

The proportion of variance explained is:

$$\mathrm{Var}(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i}.$$

33.3 Pseudocode

```
\Procedure{PCA}{Matrix M}
  Center M by subtracting mean
  Compute covariance C = (M^T M) / (n-1)
  Compute eigenvalues    and eigenvectors v of C
  Sort    in descending order
  return eigenpairs (  ,v)
\EndProcedure
```

33.4 Python code

```
import numpy as np
import matplotlib.pyplot as plt

def run_pca(matrix, out_dir=".", label="pca"):
    X = matrix - np.mean(matrix, axis=0)
    C = np.cov(X, rowvar=False)
    vals, vecs = np.linalg.eigh(C)
    idx = np.argsort(vals)[::-1]
    vals, vecs = vals[idx], vecs[:,idx]
    explained = np.cumsum(vals) / np.sum(vals)
    plt.plot(np.arange(1,len(explained)+1), explained, marker='o')
    plt.xlabel("Number of Components")
    plt.ylabel("Cumulative Variance Explained")
    plt.title(f"PCA Scree Plot ({label})")
    out_path = f"{out_dir}/{label}_pca.png"
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return vals, vecs, explained, out_path
```

33.5 Interpretation

If variance is explained by few components, the ontology collapses into a small number of latent dimensions. If many components are needed, categories and elements remain distributed.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 34
Element-Level PCA

Abstract

This chapter applies Principal Component Analysis (PCA) to the element-level projection of an ontology. Each element is represented by a binary vector indicating its category memberships. PCA identifies latent axes of variation, revealing the extent to which elements are redundantly assigned to multiple categories. A steep variance concentration indicates that element structure is captured by only a few underlying dimensions, signaling redundancy [1, 4].

34.1 Mathematical formulation

Let $M \in \{0,1\}^{m \times n}$ be the incidence matrix, where m is the number of elements and n is the number of sets. Each row r_i corresponds to the membership vector of element e_i:

$$r_i = (M_{i1}, M_{i2}, \ldots, M_{in}).$$

Centering the rows gives X, and the element covariance matrix is:

$$C_e = \frac{1}{n-1} X X^\top.$$

Eigen-decomposition yields [4]:

$$C_e v_k = \lambda_k v_k, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq 0.$$

The proportion of variance explained by the first k components is:

$$\mathrm{Var}_e(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{m} \lambda_i}.$$

34.2 Pseudocode

```
\Procedure{ElementLevelPCA}{Incidence matrix M}
  for each element e:
    row = category membership vector of e
  Center rows     X
  C = covariance(X)
  (  ,v) = eigen-decompose(C)
  Sort    in descending order
  Compute cumulative variance explained
  return   , v, cumulative variance
\EndProcedure
```

34.3 Python implementation

```
def element_level_pca(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for S in ontology_sets] for e in elements])
    vals, vecs, explained, path = run_pca(mat, out_dir, label="elements")
    return vals, explained, path
```

34.4 Analysis

- If the first principal component explains most variance, elements cluster along a single hidden axis, suggesting redundancy.
- If variance is spread across multiple components, element distribution retains complexity and independence.
- Sharp drop-offs in eigenvalues reveal latent dimensions that dominate element assignments.

34.5 Interpretation

Element-level PCA diagnoses whether semantic variety exists among elements [4]:

- **Few dominant PCs:** elements are highly redundant, their memberships reducible to only a few latent categories.
- **Many PCs needed:** elements preserve uniqueness, maintaining structural independence.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 35
Set-Level PCA

Abstract

This chapter applies Principal Component Analysis (PCA) to the set-level projection of an ontology. Each set (category) is represented by a binary vector across elements. PCA identifies latent dimensions of overlap among categories, revealing whether the ontology's classification scheme collapses into fewer underlying axes. A steep variance concentration implies redundancy among categories [1, 4].

35.1 Mathematical formulation

Let $M^\top \in \{0,1\}^{n\times m}$ represent the ontology at the set level, where each row corresponds to a set S_i across all m elements. Centering the rows gives Y, and the set covariance matrix is:

$$C_s = \frac{1}{m-1} YY^\top.$$

Eigen-decomposition yields [4]:

$$C_s u_k = \mu_k u_k, \quad \mu_1 \geq \mu_2 \geq \cdots \geq 0.$$

The variance explained by the first k principal components is:

$$\mathrm{Var}_s(k) = \frac{\sum_{i=1}^{k} \mu_i}{\sum_{i=1}^{n} \mu_i}.$$

35.2 Pseudocode

```
\Procedure{SetLevelPCA}{Incidence matrix M}
  for each set S:
    row = element membership vector of S
  Center rows     Y
  C = covariance(Y)
  (  ,u) = eigen-decompose(C)
  Sort    in descending order
  Compute cumulative variance explained
  return   , u, cumulative variance
\EndProcedure
```

35.3 Python implementation

```
def set_level_pca(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for e in elements] for S in ontology_sets])
    vals, vecs, explained, path = run_pca(mat, out_dir, label="sets")
    return vals, explained, path
```

35.4 Analysis

- A steep scree plot suggests that categories collapse into a few latent axes, implying redundancy of classification.
- Flat scree curves suggest independence across categories.
- The number of components required to explain a target proportion of variance (e.g., 90%) quantifies the effective dimensionality of the ontology.

35.5 Interpretation

Set-level PCA diagnoses the strength of the classification system [1, 4]:

- **Few dominant PCs:** categories overlap excessively and can be reduced to fewer latent dimensions.
- **Many PCs required:** categories remain distinct, resisting dimensional collapse.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 36
Relating Element-Level and Set-Level PCA

Abstract

This chapter examines the relationship between element-level and set-level Principal Component Analysis (PCA). Although both are derived from the same incidence matrix, they emphasize different perspectives: the distribution of elements across categories (row space) and the overlap of categories across elements (column space). This dual analysis enables the identification of asymmetries in redundancy and reveals whether structural collapse occurs at the element or category level [1, 4].

36.1 Conceptual foundations

Given incidence matrix $M \in \{0,1\}^{m \times n}$, two projections arise:

- **Element-level PCA:** performed on M, focusing on how elements are distributed across sets (row variance).
- **Set-level PCA:** performed on $M^\top$, focusing on how sets overlap across elements (column variance).

Although these two decompositions share the same nonzero eigenvalues, they differ in interpretation.

36.2 Mathematical relationship

The covariance matrices are:

$$C_e = MM^\top, \quad C_s = M^\top M.$$

Both matrices have the same nonzero eigenvalues $\{\lambda_i\}$, but their eigenvectors live in different spaces (elements vs. sets). Thus, variance concentration in one does not imply the same interpretation in the other.

36.3 Pseudocode

```
\Procedure{ComparePCA}{Incidence matrix M}
  ( _e , v_e) = PCA(M)          # element space
  ( _s , v_s) = PCA(M^T)        # set space
  Plot cumulative variance curves for both
  Compare number of PCs required for target variance
  Highlight asymmetries
  return Comparison report
\EndProcedure
```

36.4 Python illustration

```
def compare_pca(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    mat_elements = np.array([[1 if e in S else 0 for S in ontology_sets] for e in
        elements])
    mat_sets = mat_elements.T

    vals_e, _, explained_e, _ = run_pca(mat_elements, out_dir, label="elements")
    vals_s, _, explained_s, _ = run_pca(mat_sets, out_dir, label="sets")

    plt.plot(range(1,len(explained_e)+1), explained_e, 'b-', label="Elements PCA")
    plt.plot(range(1,len(explained_s)+1), explained_s, 'r--', label="Sets PCA")
    plt.xlabel("Number of Components")
    plt.ylabel("Cumulative Variance Explained")
    plt.title("Comparison of Element vs. Set PCA")
    plt.legend()
    out_path = os.path.join(out_dir,"pca_comparison.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return explained_e, explained_s, out_path
```

36.5 Analysis

- If both element and set PCA show rapid variance concentration, redundancy exists across both elements and categories.
- If element PCA collapses faster than set PCA, elements are overloaded but categories retain structure.
- If set PCA collapses faster than element PCA, categories are redundant while elements remain differentiated.
- If both require many components, the ontology is structurally diverse and avoids collapse.

36.6 Resolution

Asymmetries between element- and set-level PCA curves indicate the locus of redundancy in the ontology:

- **Element PCA collapse only:** semantic overloading of elements — many categories rely on the same elements.
- **Set PCA collapse only:** category collapse — categories heavily overlap in membership.
- **Both collapse:** systemic redundancy — ontology effectively reduces to a few latent dimensions.
- **Neither collapse:** balanced structure with distributed variance.

This comparative analysis therefore provides a diagnostic framework for detecting the level at which ontological redundancy is concentrated.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Part X
Principal Component Analysis - The Scree Test

Chapter 37
PCA Scree Analysis

Abstract

This chapter introduces the concept of scree analysis in the context of Principal Component Analysis (PCA). By examining the eigenvalue spectrum of a covariance or correlation matrix, scree plots reveal the number of dominant latent dimensions required to represent the data. Scree analysis is a diagnostic tool for identifying redundancy and dimensional collapse in structured systems [3, 4].

37.1 Historical perspective

Principal Component Analysis was introduced by Pearson (1901) [1] and later reformulated in modern matrix algebra terms by Hotelling (1933). The scree test was proposed by Cattell (1966) [3], who visualized eigenvalues in descending order and identified the "elbow" point as the cutoff between significant components and residual noise. Jolliffe (2002) [4] extended the method into formalized criteria and popularized its use in applied statistics. Today, scree plots remain central in dimensionality reduction, machine learning, and ontology evaluation.

37.2 Conceptual foundations

Given a dataset represented by an incidence or data matrix M, PCA decomposes its variance into orthogonal components. The eigenvalue spectrum encodes how much variance each component explains.

- Large eigenvalues correspond to dominant latent dimensions.
- Small eigenvalues correspond to noise or residual variation.
- The scree plot visualizes the ordered eigenvalues $\lambda_1, \ldots, \lambda_p$.

The "elbow criterion" defines the cutoff where variance contributions flatten.

37.3 Mathematical formulation

Let X be a mean-centered data matrix. The covariance matrix is:

$$C = \frac{1}{n-1} X^\top X.$$

Eigen-decomposition yields:

$$Cv_k = \lambda_k v_k, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq 0.$$

The scree plot is the function:

$$f(k) = \lambda_k, \quad k = 1, \ldots, p.$$

37.4 Pseudocode

```
\Procedure{ScreePCA}{Data matrix X}
  Center columns of X
  C = covariance(X)
  (  ,v) = eigen-decompose(C)
  Sort    in descending order
  Plot  _i  vs i
  return
\EndProcedure
```

37.5 Python illustration

```python
def scree_analysis(data_matrix, out_path="scree.png"):
    # center data
    X = data_matrix - np.mean(data_matrix, axis=0)
    cov = np.cov(X, rowvar=False)
    vals, vecs = np.linalg.eigh(cov)
    vals = np.flip(np.sort(vals))  # descending order

    plt.plot(range(1, len(vals)+1), vals, marker="o")
    plt.xlabel("Principal Component")
    plt.ylabel("Eigenvalue")
    plt.title("PCA Scree Plot")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return vals, out_path
```

37.6 Analysis and interpretation

- **Steep scree curve:** variance dominated by a few components; system collapses into fewer latent dimensions.
- **Flat scree curve:** variance distributed across many components; system preserves structural independence.
- **Elbow test:** cutoff point between dominant and residual variance.

37.7 Conclusion

Scree analysis provides a simple yet powerful diagnostic for ontological and data structures. It reveals whether a classification system is effectively low-dimensional (collapsed) or retains distributed variance across multiple dimensions. As such, it bridges dimensionality reduction, redundancy detection, and structural evaluation.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[3] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 38
Element-Level PCA Scree Analysis

Abstract

This chapter applies scree analysis to the element-level projection of an ontology. By examining the eigenvalue spectrum of the covariance matrix, we assess how many latent dimensions are required to capture the structural variability of element memberships. Steep drops in the scree plot indicate dimensional collapse, while flat curves suggest distributed structural diversity [3, 4].

38.1 Historical perspective

Principal Component Analysis (PCA) was first introduced by Karl Pearson in 1901 [1] as a method of finding lines and planes of closest fit to systems of points in space. The scree test itself was introduced later by Raymond Cattell (1966) [3], who proposed that plotting eigenvalues in descending order reveals a natural "elbow" where significant components end and noise begins. Later refinements by Jolliffe [4] provided statistical rigor and applications across multiple domains, cementing scree analysis as a core tool for dimensionality reduction diagnostics.

38.2 Mathematical formulation

Let $M \in \{0,1\}^{m \times n}$ be the incidence matrix. The element covariance matrix is:

$$C_e = \frac{1}{n-1} X X^\top, \quad X = \text{centered}(M).$$

Eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_m \geq 0$ represent variances captured by successive principal components. The scree plot graphs λ_i against component index i.

38.3 Pseudocode

```
\Procedure{ElementScreePCA}{Incidence matrix M}
  Center rows     X
  C = covariance(X)
  (  ,v) = eigen-decompose(C)
  Sort    in descending order
  Plot scree curve  _i  vs i
  return
\EndProcedure
```

38.4 Python implementation

```
def element_scree_pca(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for S in ontology_sets] for e in elements])
    vals, _, _, path = run_pca(mat, out_dir, label="element_scree")
    return vals, path
```

38.5 Analysis and interpretation

- A single dominant eigenvalue indicates redundancy: many elements cluster along a single hidden axis.
- A gradual decline indicates structural richness with multiple independent dimensions.
- The elbow criterion [3] identifies the effective number of latent dimensions.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[3] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 39
Set-Level PCA Scree Analysis

Abstract

This chapter applies scree analysis to the set-level projection of an ontology. Each set is represented by a binary vector across elements. The eigenvalue spectrum of the covariance matrix reveals whether categories collapse into fewer latent dimensions or preserve structural diversity [3, 4].

39.1 Historical perspective

The idea of analyzing category overlap using PCA derives from Pearson's formulation of orthogonal axes of variance [1]. Cattell's (1966) scree method [3] provided a diagnostic tool for identifying meaningful versus noise-driven components, and was later expanded into full dimensionality selection frameworks. Set-level PCA generalizes these principles by treating categories instead of individuals as the units of analysis, but the conceptual roots trace back to early psychometrics and multivariate analysis.

39.2 Mathematical formulation

Let $M^\top \in \{0,1\}^{n\times m}$. The set covariance matrix is:

$$C_s = \frac{1}{m-1} YY^\top, \quad Y = \text{centered}(M^\top).$$

Eigenvalues $\mu_1 \geq \mu_2 \geq \cdots \geq \mu_n \geq 0$ capture variance.

39.3 Pseudocode

```
\Procedure{SetScreePCA}{Incidence matrix M}
  Transpose M     M^T
  Center rows     Y
  C = covariance(Y)
  (  ,u) = eigen-decompose(C)
  Sort    in descending order
  Plot scree curve  _i  vs i
  return
\EndProcedure
```

39.4 Python implementation

```
def set_scree_pca(ontology_sets, out_dir="."):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for e in elements] for S in ontology_sets])
    vals, _, _, path = run_pca(mat, out_dir, label="set_scree")
    return vals, path
```

39.5 Analysis and interpretation

- A steep scree drop indicates redundancy among categories.
- Flat eigenvalue spectra suggest independence across categories.
- The elbow test helps identify the number of underlying latent axes.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[3] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 40
Interrelationships: Element vs. Set PCA Scree Analysis

Abstract

This chapter compares scree plots from element- and set-level PCA, highlighting asymmetries in variance concentration. Although the two covariance matrices share the same nonzero eigenvalues, their interpretation differs across the element and set perspectives. Understanding this duality is crucial for identifying the locus of redundancy in ontological analysis [1, 4].

40.1 Historical perspective

The duality between $MM^\top$ and $M^\top M$ traces to linear algebra foundations in spectral theory, but its use in multivariate data analysis was pioneered by Pearson (1901) [1]. Later, Cattell's scree method [3] extended this into a practical diagnostic tool, while Jolliffe's work [4] clarified how eigenvalue spectra reflect redundancy. Applying this dual perspective to ontologies extends these classical tools into knowledge system evaluation.

40.2 Mathematical relationship

For incidence matrix $M \in \{0,1\}^{m\times n}$:

$$C_e = MM^\top, \quad C_s = M^\top M.$$

Both have the same eigenvalues, but eigenvectors differ.

40.3 Pseudocode

```
\Procedure{CompareScree}{Incidence matrix M}
     = eigenvalues(M M^T)  # element scree
     = eigenvalues(M^T M)  # set scree
  Normalize   ,
  Plot both curves
  Compare steepness, elbows
  return comparison
\EndProcedure
```

40.4 Python illustration

```
def compare_scree(ontology_sets, out_dir="."):
    vals_e, _ = element_scree_pca(ontology_sets, out_dir)
    vals_s, _ = set_scree_pca(ontology_sets, out_dir)
    ...
```

40.5 Resolution

- Element scree collapse → redundancy in elements.
- Set scree collapse → redundancy in categories.
- Both collapse → systemic redundancy.
- Neither collapse → balanced structure.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.

[3] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 41
Interrelationships: PCA and Scree Analysis

Abstract

This chapter clarifies the relationship between Principal Component Analysis (PCA) in general and scree analysis in particular. PCA is the broader methodology of decomposing covariance structure into orthogonal components, while scree analysis is a diagnostic tool that interprets the eigenvalue spectrum to identify the effective dimensionality of the system. Understanding this relationship ensures that scree plots are interpreted within the broader mathematical and historical context of PCA [1, 2, 3, 4].

41.1 Historical perspective

PCA was introduced by Pearson (1901) [1] and reformulated as a multivariate statistical technique by Hotelling (1933) [2]. Cattell (1966) [3] introduced the scree test as a visual tool for determining the number of significant principal components. Jolliffe (2002) [4] provided refinements for threshold-based and variance-based selection. The scree test thus represents a specialization within PCA's broader framework.

41.2 Conceptual relationship

- PCA produces both eigenvalues and eigenvectors of the covariance matrix.
- Scree analysis uses only the eigenvalues, plotting them in descending order to visualize variance concentration.
- PCA without scree provides complete data transformation but lacks intuitive diagnostics of dimensionality.
- Scree without PCA context reduces to a curve with no interpretive grounding in variance decomposition.

41.3 Mathematical linkage

Given covariance matrix $C = \frac{1}{n-1} X^\top X$, PCA solves:

$$Cv_k = \lambda_k v_k.$$

The full PCA produces eigenpairs (λ_k, v_k) for $k = 1, \ldots, p$. Scree analysis uses only:

$$\{\lambda_1, \lambda_2, \ldots, \lambda_p\},$$

arranged in descending order. Thus, scree is a projection of PCA's results onto the eigenvalue spectrum.

41.4 Pseudocode

```
\Procedure{PCA\_with\_Scree}{Data matrix X}
  Center X
  C = covariance(X)
  (  ,v) = eigen-decompose(C)
  Plot  _i  vs i  # scree curve
  return   , v, scree
\EndProcedure
```

41.5 Python illustration

```
def pca_with_scree(data_matrix, out_dir="."):
    X = data_matrix - np.mean(data_matrix, axis=0)
    cov = np.cov(X, rowvar=False)
    vals, vecs = np.linalg.eigh(cov)
    vals = np.flip(np.sort(vals))
    # scree plot
    plt.plot(range(1,len(vals)+1), vals, marker="o")
    plt.title("PCA with Scree Plot")
    out_path = os.path.join(out_dir,"pca_with_scree.png")
    plt.savefig(out_path,dpi=300,bbox_inches="tight")
    plt.close()
    return vals, vecs, out_path
```

41.6 Analysis

- When PCA eigenvectors reveal clear axes and the scree plot confirms a sharp elbow, conclusions are consistent: a few dominant latent dimensions explain the system.
- When PCA suggests high-dimensional structure but the scree curve flattens quickly, redundancy may be overstated.
- When scree indicates many dimensions but eigenvectors cluster semantically, structure may still be reducible.

41.7 Resolution

If PCA and scree analysis diverge:

- **Scree collapse, PCA structure:** scree oversimplifies, interpret eigenvectors for nuance.
- **PCA rich, scree flat:** variance is thinly spread; scree is correct that no strong latent axes dominate.
- **Both collapse:** true redundancy exists.
- **Both rich:** structural independence is confirmed.

Thus, scree should always be interpreted as a complementary projection of PCA.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.
[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Part XI
Principal Component Analysis - Loadings on Components

Chapter 42
Principal Component Loadings

Abstract

This chapter introduces the concept of loadings in Principal Component Analysis (PCA). While eigenvalues quantify the amount of variance explained by each principal component, loadings represent the correlation or contribution of each original variable to the new components. Loadings thus provide interpretive meaning, enabling the mapping of latent dimensions back to the ontology's original variables [2, 4].

42.1 Historical perspective

While PCA was first introduced by Pearson (1901) [1], the interpretation of component loadings as a way of mapping factors onto original variables was systematized by Hotelling (1933) [2]. Later refinements by Harman (1976) and Jolliffe (2002) emphasized the role of loadings in interpretability, distinguishing them from the purely mathematical eigenvectors by normalizing them in relation to the original data space. Today, loadings are central to the application of PCA in factor analysis, psychology, and ontology evaluation.

42.2 Conceptual foundations

PCA decomposes a data matrix X into scores and loadings:

$$X = TP^\top,$$

where:

- T are the principal component scores (projection of samples).
- P are the loadings (directions of maximum variance).

If v_k is the eigenvector of covariance matrix C, the loading of variable j on component k is:

$$L_{jk} = v_{jk}\sqrt{\lambda_k}.$$

Loadings are thus scaled eigenvectors, interpretable as correlations between original variables and principal components.

42.3 Mathematical formulation

Given covariance matrix $C = \frac{1}{n-1}X^\top X$, PCA solves:

$$Cv_k = \lambda_k v_k.$$

Scores are:

$$t_k = X v_k,$$

and loadings are:

$$L_{jk} = \frac{X_j^\top t_k}{\|t_k\|^2}.$$

In practice, loadings can be computed as the eigenvectors scaled by the square root of eigenvalues:

$$L = V \Lambda^{1/2}.$$

42.4 Pseudocode

```
\Procedure{PCALoadings}{Data matrix X}
  Center X
  C = covariance(X)
  (  ,V) = eigen-decompose(C)
  for each eigenpair ( _k , v_k):
    L_k = v_k * sqrt( _k )
  return L
\EndProcedure
```

42.5 Python illustration

```
def pca_loadings(data_matrix):
    X = data_matrix - np.mean(data_matrix, axis=0)
    cov = np.cov(X, rowvar=False)
    vals, vecs = np.linalg.eigh(cov)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:,order]
    loadings = vecs * np.sqrt(vals)
    return loadings, vals
```

42.6 Analysis and interpretation

- Large positive or negative loadings indicate strong contributions of variables to components.
- Near-zero loadings indicate weak or irrelevant contributions.
- Patterns of loadings across variables reveal latent structures, such as clusters of variables mapping to the same component.
- Interpretation depends on comparing loadings across components rather than in isolation.

42.7 Conclusion

Loadings provide the interpretive bridge between abstract principal components and the original ontology. They show which variables (elements or sets) define the latent dimensions revealed by PCA, complementing the variance-focused perspective of scree analysis.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.
[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Harman, H. H. (1976). *Modern Factor Analysis* (3rd ed.). University of Chicago Press.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 43
Element-Level PCA Loadings

Abstract

This chapter applies loadings analysis to the element-level projection of an ontology. Each element is represented by its contributions to principal components, revealing which elements define the dominant latent dimensions. Loadings highlight redundancy or uniqueness at the element level, showing whether certain elements are overrepresented across categories [2, 3, 4].

43.1 Historical perspective

Hotelling (1933) [2] formalized the concept of component loadings as the link between eigenvectors and the original variables. Later developments in factor analysis, especially Harman's (1976) [3] treatment, emphasized their interpretive power. Jolliffe (2002) [4] refined methods of normalization and rotation, but the essence remains: loadings map latent dimensions back onto variables, here applied to elements.

43.2 Mathematical formulation

Let $M \in \{0,1\}^{m \times n}$ be the incidence matrix. Element covariance:

$$C_e = MM^\top.$$

Eigenpairs:

$$C_e v_k = \lambda_k v_k.$$

Loadings of element i on component k:

$$L_{ik} = v_{ik}\sqrt{\lambda_k}.$$

43.3 Pseudocode

```
\Procedure{ElementLoadings}{Incidence matrix M}
  C = M M^T
  (  ,V) = eigen-decompose(C)
  for each k:
    for each element i:
      L[i,k] = V[i,k] * sqrt(  [k])
  return L
\EndProcedure
```

43.4 Python illustration

```
def element_loadings(ontology_sets):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for S in ontology_sets] for e in elements])
    cov = np.cov(mat, rowvar=True)
    vals, vecs = np.linalg.eigh(cov)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:,order]
    loadings = vecs * np.sqrt(vals)
    return elements, loadings
```

43.5 Analysis and interpretation

- Elements with high loadings on the first component are heavily shared across categories (redundancy).
- Elements with strong loadings on secondary components reflect specialized or differentiating features.
- Loadings near zero indicate peripheral or non-informative elements.

References

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Harman, H. H. (1976). *Modern Factor Analysis* (3rd ed.). University of Chicago Press.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 44
Set-Level PCA Loadings

Abstract

This chapter applies loadings analysis to the set-level projection of an ontology. Each set is represented by its contributions to principal components, revealing which categories dominate latent axes of overlap. Set-level loadings expose redundancy among categories, identifying those that collapse into the same underlying dimensions [2, 3, 4].

44.1 Historical perspective

The interpretation of component loadings as weights of original variables on latent factors originated with Hotelling (1933) [2]. In applied multivariate analysis, Harman (1976) [3] highlighted how loadings guide the labeling of factors. Jolliffe (2002) [4] extended the approach into large-scale applications. Applied here, set-level loadings reveal redundancy at the category level.

44.2 Mathematical formulation

Let $M^\top \in \{0,1\}^{n\times m}$. Set covariance:

$$C_s = M^\top M.$$

Eigenpairs:

$$C_s u_k = \mu_k u_k.$$

Loadings of set j on component k:

$$L_{jk} = u_{jk}\sqrt{\mu_k}.$$

44.3 Pseudocode

```
\Procedure{SetLoadings}{Incidence matrix M}
  C = M^T M
  (  ,U) = eigen-decompose(C)
  for each k:
    for each set j:
      L[j,k] = U[j,k] * sqrt(  [k])
  return L
\EndProcedure
```

44.4 Python illustration

```
def set_loadings(ontology_sets):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for e in elements] for S in ontology_sets])
    cov = np.cov(mat, rowvar=True)
    vals, vecs = np.linalg.eigh(cov)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:,order]
    loadings = vecs * np.sqrt(vals)
    return loadings
```

44.5 Analysis and interpretation

- Sets with high loadings on the same component indicate redundancy — their categories collapse into the same latent dimension.
- Sets distributed across multiple components represent diverse classifications.
- Loadings close to zero suggest categories with minimal influence on the structure.

References

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Harman, H. H. (1976). *Modern Factor Analysis* (3rd ed.). University of Chicago Press.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 45
Interrelationships: Element vs. Set PCA Loadings

Abstract

This chapter examines the relationship between element-level and set-level PCA loadings. While element loadings identify which elements dominate latent components, set loadings reveal which categories collapse into the same dimensions. Together, they provide a two-sided view of redundancy: one at the level of shared features, the other at the level of overlapping categories [2, 3, 4].

45.1 Historical perspective

The dual use of loadings at both variable and group levels traces back to Hotelling's (1933) [2] formulation of PCA, expanded by Harman (1976) [3] for factor analysis, and refined by Jolliffe (2002) [4]. Applying this dual view to ontologies reveals structural redundancy either in elements or in sets.

45.2 Conceptual relationship

- Element loadings emphasize which features define the latent axes.
- Set loadings emphasize which categories collapse into those axes.
- Comparing them reveals whether redundancy originates in the elements or the categories.

45.3 Mathematical linkage

Element covariance:

$$C_e = MM^\top, \quad \text{eigenvectors } V.$$

Set covariance:

$$C_s = M^\top M, \quad \text{eigenvectors } U.$$

Loadings are $L_e = V\Lambda^{1/2}$ and $L_s = U\Lambda^{1/2}$, with the same eigenvalues Λ but different eigenvectors.

45.4 Pseudocode

```
\Procedure{CompareLoadings}{Incidence matrix M}
  L_e = element_loadings(M)
  L_s = set_loadings(M)
  Compare magnitude and structure of L_e vs L_s
  Highlight asymmetries
  return comparison
\EndProcedure
```

45.5 Python illustration

```
def compare_loadings(ontology_sets):
    elems, loadings_e = element_loadings(ontology_sets)
    loadings_s = set_loadings(ontology_sets)
    return loadings_e, loadings_s
```

45.6 Resolution

- If element loadings cluster but set loadings are dispersed, redundancy originates in the elements.
- If set loadings cluster but element loadings remain diverse, redundancy originates in the categories.
- If both cluster, systemic redundancy reduces ontology dimensionality.
- If both disperse, the ontology retains balance and structural independence.

References

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Harman, H. H. (1976). *Modern Factor Analysis* (3rd ed.). University of Chicago Press.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Chapter 46
Interrelationships: PCA and Loadings

Abstract

This chapter clarifies the relationship between Principal Component Analysis (PCA) in general and the interpretation of loadings in particular. While PCA provides eigenvalues and eigenvectors of the covariance matrix, loadings translate eigenvectors into interpretable weights that link principal components to original variables. Scree analysis emphasizes variance concentration, whereas loadings emphasize the semantic content of components [1, 2, 3, 4].

46.1 Historical perspective

Pearson (1901) [1] introduced PCA as a method for dimensionality reduction. Hotelling (1933) [2] expanded the method into multivariate statistics and formalized the interpretation of loadings as correlations between variables and principal components. Harman (1976) [3] emphasized the importance of loadings in factor analysis, while Jolliffe (2002) [4] refined loadings-based interpretive practices. Thus, PCA provides the decomposition, and loadings provide its interpretive link.

46.2 Conceptual relationship

- PCA yields eigenpairs (λ_k, v_k).
- Scree analysis interprets the eigenvalues λ_k.
- Loadings interpret the eigenvectors v_k in relation to the original variables.
- Together, scree and loadings provide complementary insights: one quantifies explained variance, the other explains structure.

46.3 Mathematical linkage

Given covariance matrix $C = \frac{1}{n-1} X^\top X$, PCA solves:

$$Cv_k = \lambda_k v_k.$$

- Scree analysis uses $\{\lambda_k\}$.
- Loadings are computed as:

$$L_{jk} = v_{jk}\sqrt{\lambda_k},$$

representing the contribution of variable j to component k.

46.4 Pseudocode

```
\Procedure{PCAwithLoadings}{Data matrix X}
  Center X
  C = covariance(X)
  (  ,V) = eigen-decompose(C)
  for each k:
    for each variable j:
      L[j,k] = V[j,k] * sqrt(  [k])
  return   , V, L
\EndProcedure
```

46.5 Python illustration

```
def pca_with_loadings(data_matrix):
    X = data_matrix - np.mean(data_matrix, axis=0)
    cov = np.cov(X, rowvar=False)
    vals, vecs = np.linalg.eigh(cov)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:,order]
    loadings = vecs * np.sqrt(vals)
    return vals, vecs, loadings
```

46.6 Analysis

- PCA provides both eigenvalues (variance explained) and eigenvectors (directions).
- Scree interprets eigenvalues: how many dimensions matter.
- Loadings interpret eigenvectors: what those dimensions mean.
- Together they resolve both the "quantity" and the "quality" of latent dimensions.

46.7 Resolution

When PCA, scree, and loadings differ:

- **Strong scree, weak loadings:** dimensionality is clear but interpretability is poor.
- **Weak scree, strong loadings:** dimensions explain little variance but remain semantically meaningful.
- **Both strong:** a robust latent structure is confirmed.
- **Both weak:** redundancy and collapse dominate.

Thus, PCA should always be read with both scree and loadings together.

References

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572.
[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
[3] Harman, H. H. (1976). *Modern Factor Analysis* (3rd ed.). University of Chicago Press.
[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

Volume V

Information Retrieval / Library Science

Part XII
Overlap Index

Chapter 47
Overlap Index (General)

Abstract

The Overlap Index (OI) quantifies the proportion of shared elements relative to the size of the intersecting sets. Unlike Jaccard similarity, which normalizes by the union, OI normalizes by the minimum or average set size, thereby emphasizing redundancy relative to category scope. This chapter introduces the general framework for OI, situating it historically and mathematically.

47.1 Introduction

Overlap Index highlights the extent to which sets are indistinguishable due to high shared membership. As such, it provides a lens for diagnosing whether categories are genuinely independent or functionally collapsed.

47.2 Historical Perspective

The origins of overlap analysis lie in early probability theory, where Laplace and others considered overlapping events in sample spaces. In information retrieval, Salton emphasized overlap measures as indicators of redundancy in document indexing [1]. Later, ontology engineering adopted overlap measures to evaluate taxonomic independence [3]. OI formalizes this tradition into a simple, interpretable metric.

47.3 Conceptual Foundations

Given two sets A and B, OI captures the degree of overlap normalized by set sizes. It is designed to reveal redundancy beyond mere pairwise intersection counts.

47.4 Mathematical Formalism

A common definition is:

$$OI(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

Where: - $OI = 1$ indicates one set is entirely contained in the other. - $OI = 0$ indicates disjoint sets.

47.5 Pseudocode

```
Input: Two sets A, B
intersection = |A     B|
denominator = min(|A|, |B|)
if denominator = 0:
    return 0
OI = intersection / denominator
Output OI
```

47.6 Python Algorithm

```python
def overlap_index(A, B):
    if not A or not B:
        return 0
    return len(A & B) / min(len(A), len(B))
```

47.7 Analysis and Interpretation

- $OI = 1$: complete containment, categories collapse. - $0.5 < OI < 1$: significant redundancy. - $OI \approx 0$: distinct categories.

47.8 Significance

OI reveals whether categories provide distinct informational value or whether one is subsumed into another. It is more sensitive than Jaccard when category sizes differ.

47.9 Conclusion

As a diagnostic tool, OI is central to ontology autopsy, revealing functional duplication among categories.

References

References

[1] Salton, G. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.
[3] Brank, J., Grobelnik, M., & Mladenić, D. (2005). A survey of ontology evaluation techniques. In *Proceedings of the Conference on Data Mining and Data Warehouses (SiKDD)*.

Chapter 48
Overlap Index: Element-Level Analysis

Abstract

Element-level OI measures the extent to which specific elements are responsible for overlaps across sets. This chapter introduces element-centric OI, focusing on micro-level redundancy patterns.

48.1 Introduction

Whereas general OI measures category overlap, element-level OI identifies which elements contribute disproportionately to overlaps.

48.2 Historical Perspective

The problem of "ambiguous elements" dates back to Aristotelian logic, where terms used across categories blurred genus-differentia distinctions [1]. In bibliometrics, Garfield noted that certain "hub terms" drove overlap across subject domains [2]. Element-level OI continues this tradition by quantifying the degree of element-driven overlap.

48.3 Conceptual Foundations

For each element e, compute its participation in overlapping pairs of sets.

48.4 Mathematical Formalism

$$OI_{element}(e) = \frac{1}{\binom{n}{2}} \sum_{i<j} \mathbf{1}_{\{e \in S_i \cap S_j\}}$$

48.5 Pseudocode

```
For each element e in U:
   overlap_count = 0
   For each pair (Si, Sj):
       if e in both:
           overlap_count += 1
OI_element(e) = overlap_count / total_pairs
```

48.6 Python Algorithm

```python
def element_level_oi(ontology_sets):
    from itertools import combinations
    U = set().union(*ontology_sets)
    total_pairs = len(ontology_sets) * (len(ontology_sets) - 1) / 2
    scores = {}
    for e in U:
        count = 0
        for A, B in combinations(ontology_sets, 2):
            if e in A and e in B:
                count += 1
        scores[e] = count / total_pairs if total_pairs else 0
    return scores
```

48.7 Analysis

- High $OI_{element}$: overstretched elements. - Low $OI_{element}$: specialized elements.

48.8 Significance

Element-level OI pinpoints sources of ontological collapse at micro-scale.

48.9 Conclusion

This analysis clarifies whether redundancy is caused by particular elements rather than structural category overlap.

References

References

[1] Aristotle. (350 BCE). *Categories*.
[2] Garfield, E. (1979). *Citation Indexing: Its Theory and Application*. Wiley.

Chapter 49
Overlap Index: Set-Level Analysis

Abstract

Set-level OI captures the degree of overlap between categories relative to their sizes, emphasizing macro-structural redundancy. This chapter develops the formalism and interpretation of set-level OI.

49.1 Introduction

Unlike element-level analysis, set-level OI evaluates redundancy between categories as wholes. It identifies whether sets are structurally distinct or functionally redundant.

49.2 Historical Perspective

In library classification systems, subject headings often overlapped to the point of redundancy [1]. Measures of overlap were introduced to optimize subject taxonomies. Set-level OI formalizes this into a systematic metric for ontologies.

49.3 Conceptual Foundations

For two sets S_i, S_j, compute $OI(S_i, S_j)$ using intersection normalized by smaller set size. Global set-level OI is obtained by averaging across all pairs.

49.4 Mathematical Formalism

$$OI_{set} = \frac{2}{n(n-1)} \sum_{i<j} \frac{|S_i \cap S_j|}{\min(|S_i|, |S_j|)}$$

49.5 Pseudocode

```
total = 0
count = 0
For each pair (Si, Sj):
    if min(|Si|,|Sj|) > 0:
        total += |Si     Sj| / min(|Si|,|Sj|)
        count += 1
OI_set = total / count
```

49.6 Python Algorithm

```python
def set_level_oi(ontology_sets):
    from itertools import combinations
    scores = []
    for A, B in combinations(ontology_sets, 2):
        if min(len(A), len(B)) > 0:
            scores.append(len(A & B) / min(len(A), len(B)))
    return sum(scores)/len(scores) if scores else 0
```

49.7 Analysis

- $OI_{set} \approx 1$: categories collapse. - Moderate values: partial redundancy. - $OI_{set} \approx 0$: structural independence.

49.8 Significance

This metric reveals macro-level structural inefficiencies.

49.9 Conclusion

Set-level OI diagnoses redundancy between categories in aggregate.

References

References

[1] Beghtol, C. (1986). Semantic validity: Concepts of warrant in bibliographic classification systems. *Library Resources & Technical Services*, 30(2), 109–125.

Chapter 50
Overlap Index: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter examines the relationship between element-level and set-level OI, highlighting scenarios where the two diverge. A resolution framework is proposed.

50.1 Introduction

While both perspectives quantify redundancy, they emphasize different levels of granularity. Their interrelationship provides insight into whether redundancy arises from specific elements or from structural overlap of categories.

50.2 Historical Perspective

The micro–macro tension in classification was noted in Durkheim's sociology, where individual facts did not always scale to collective structures [1]. In thesaurus construction, element-level synonym overlaps often translated into redundant category structures [2]. OI continues this dual-level analysis.

50.3 Conceptual Foundations

- Element-level OI: focuses on overstretched elements. - Set-level OI: focuses on redundant categories.

50.4 Analysis and Interpretation

- If $OI_{element}$ and OI_{set} both high: redundancy systemic. - If $OI_{element}$ high but OI_{set} low: redundancy localized in few elements. - If OI_{set} high but $OI_{element}$ low: structural duplication across categories.

50.5 Resolution

Discrepancies reveal whether ontology requires refinement at the element or category level.

50.6 Conclusion

Comparing both provides a diagnostic lens that distinguishes micro-level from macro-level redundancy.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.
[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

Part XIII
Membership Histogram Analysis

Chapter 51
Interrelationships: Element vs. Set Membership Histograms

Abstract

This chapter examines the relationship between element-level and set-level membership histograms. Together, they reveal whether redundancy originates primarily in individual elements or in sets themselves. Their comparison resolves ambiguities in interpreting overlap and collapse [2, 1].

51.1 Conceptual relationship

- Element histogram: counts how many sets each element belongs to.
- Set histogram: counts how many elements each set contains.
- Comparing them shows whether redundancy is element-driven (cross-linking features) or set-driven (dominant categories).

51.2 Resolution

- If element histogram shows high redundancy but set histogram is balanced → overlap originates from multi-assigned elements.
- If set histogram is skewed but element histogram is balanced → redundancy originates from large, dominant sets.
- If both are skewed → systemic collapse.

References

[2] Smith, B. (2008). Ontology (science). In *Formal Ontology in Information Systems*, IOS Press.
[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

Chapter 52
Element-Level Membership Histogram

Abstract

At the element level, membership histograms track how many sets each element participates in. This highlights redundancy or uniqueness of individual elements and identifies which elements act as cross-links across multiple categories [2, 1].

52.1 Mathematical formulation

For element i:

$$d_i = \sum_{j=1}^{n} M_{ij}.$$

Histogram $\{d_i\}$ reveals whether elements are unique ($d_i = 1$) or overlapping ($d_i > 1$).

52.2 Pseudocode

```
\Procedure{ElementHistogram}{M}
  for each element i:
    d[i] = sum_j M[i,j]
  histogram(d)
\EndProcedure
```

52.3 Python illustration

```python
def element_histogram(ontology_sets):
    elements = sorted(set().union(*ontology_sets))
    degrees = [sum(1 for S in ontology_sets if e in S) for e in elements]
    return degrees
```

52.4 Analysis

- Elements with $d_i = 1$: unique identifiers, healthy ontology.
- Elements with $d_i > 3$: signals redundancy and collapse risk.
- Outliers (extremely high d_i) act as "universal" elements, undermining category distinction.

References

[2] Smith, B. (2008). Ontology (science). In *Formal Ontology in Information Systems*, IOS Press.
[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

Chapter 53
Set-Level Membership Histogram

Abstract

At the set level, the histogram measures how many elements are shared across sets. This provides a complementary view, focusing not on element redundancy but on category redundancy — which sets overlap heavily and which remain distinct [1].

53.1 Mathematical formulation

For set j:

$$s_j = \sum_{i=1}^{m} M_{ij},$$

the number of elements in set j. Comparing $\{s_j\}$ across sets shows distribution of set sizes and their potential dominance.

53.2 Pseudocode

```
\Procedure{SetHistogram}{M}
  for each set j:
    s[j] = sum_i M[i,j]
  histogram(s)
\EndProcedure
```

53.3 Python illustration

```python
def set_histogram(ontology_sets):
    sizes = [len(S) for S in ontology_sets]
    return sizes
```

53.4 Analysis

- Narrow size distribution: balanced categories.
- Wide distribution: dominance of some sets, underrepresentation of others.
- Extremely large sets → ontological inflation; extremely small sets → fragmentation.

References

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

Chapter 54
Membership Histogram

Abstract

The membership histogram counts how many sets (categories) each element belongs to, forming a distribution that reflects redundancy and overlap. This diagnostic provides a basic yet powerful way to visualize whether elements are unique to categories or shared across multiple categories. It is a core tool in ontology evaluation, linking incidence structures to measures of orthogonality and redundancy [1, 2].

54.1 Historical perspective

The idea of counting membership overlaps originates in set theory (Cantor, 1895 [1]) and was later applied to taxonomy evaluation and ontology engineering (Smith, 2008 [2]; Guarino et al., 2009 [1]). Histograms are now a standard diagnostic in ontology validation pipelines.

54.2 Conceptual foundations

For ontology incidence matrix $M \in \{0,1\}^{m \times n}$:

$$d_i = \sum_{j=1}^{n} M_{ij},$$

where d_i is the degree of element i (number of sets it belongs to). The histogram of $\{d_i\}$ summarizes redundancy distribution.

54.3 Pseudocode

```
\Procedure{MembershipHistogram}{Incidence matrix M}
  for each element i:
    d[i] = sum_j M[i,j]
  Plot histogram of d
  return d
\EndProcedure
```

54.4 Python illustration

```
def membership_histogram(ontology_sets, out_path="membership_hist.png"):
    elements = sorted(set().union(*ontology_sets))
    degrees = [sum(1 for S in ontology_sets if e in S) for e in elements]
    plt.hist(degrees, bins=range(1, max(degrees)+2))
    plt.title("Membership Histogram")
    plt.xlabel("Number of Sets per Element")
    plt.ylabel("Frequency")
    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    return degrees
```

54.5 Analysis

- Narrow histogram with peak at 1 → orthogonal sets, minimal redundancy.
- Wide histogram with heavy tail → significant overlap, risk of collapse.
- Bimodal or skewed distributions → partial redundancy structures.

References

[1] Cantor, G. (1895). Beiträge zur Begründung der transfiniten Mengenlehre. *Mathematische Annalen*, 46, 481–512.

[2] Smith, B. (2008). Ontology (science). In *Formal Ontology in Information Systems*, IOS Press.

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

Volume VI

Network Science / Sociology

Part XIV
Community Detection Analysis

Chapter 55
Community Detection in Ontology Graphs

Abstract

Community detection in ontology graphs identifies clusters of categories that overlap heavily with each other. Representing sets as nodes and overlaps as weighted edges allows the use of graph algorithms to reveal redundant meta-structures, highlighting collapse or modularity in the ontology [10, 5**?**].

55.1 Historical perspective

The study of community detection originates in network science. Girvan and Newman (2002) [10] introduced the modularity framework, formalizing how to partition graphs into dense subgraphs. Newman (2006) [5] refined modularity optimization, and Fortunato (2010) [**?**] surveyed methods for complex networks. Applying these tools to ontologies uncovers hidden modular structures among sets.

55.2 Conceptual foundations

Let $G = (V, E)$ where V are sets and edges e_{ij} have weight w_{ij} equal to overlap. A community is a subset $C \subset V$ such that:

$$\frac{\sum_{i,j \in C} w_{ij}}{\sum_{i,j \in V} w_{ij}} \gg \frac{\sum_{i \in C, j \notin C} w_{ij}}{\sum_{i,j \in V} w_{ij}}.$$

55.3 Pseudocode

```
\Procedure{CommunityDetection}{Graph G}
  Build adjacency matrix A from overlaps
  Apply modularity-based clustering
  return partition of nodes into communities
\EndProcedure
```

55.4 Python illustration

```
import networkx as nx
from networkx.algorithms import community

def set_graph_communities(ontology_sets):
    n = len(ontology_sets)
    G = nx.Graph()
    for i in range(n):
        for j in range(i+1, n):
            w = len(ontology_sets[i] & ontology_sets[j])
            if w > 0:
                G.add_edge(i, j, weight=w)
    comms = community.greedy_modularity_communities(G, weight="weight")
    return comms
```

55.5 Analysis

- Large communities indicate redundancy among many sets.
- Small communities indicate localized overlap.
- Modularity close to 1 → strong community structure; close to 0 → weak or no m

Chapter 56
Element-Level Community Analysis

Abstract

Element-level community detection represents elements as nodes, with edges linking elements that co-occur in the same sets. Communities then correspond to groups of elements that frequently appear together across categories, revealing redundancy in the feature space [5?].

56.1 Mathematical formulation

Incidence matrix M: Define element–element adjacency:

$$A_e(i,j) = \sum_k M_{ik} M_{jk}.$$

Apply modularity optimization on A_e.

56.2 Pseudocode

```
\Procedure{ElementCommunities}{M}
  A_e = M M^T
  Build graph from A_e
  Detect communities
  return partition of elements
\EndProcedure
```

56.3 Python illustration

```python
def element_communities(ontology_sets):
    elements = sorted(set().union(*ontology_sets))
    mat = np.array([[1 if e in S else 0 for S in ontology_sets] for e in elements
        ])
    A = mat @ mat.T
    G = nx.from_numpy_array(A)
    comms = community.greedy_modularity_communities(G)
    return comms
```

56.4 Analysis

- Communities identify clusters of elements acting as “building blocks” across categories.
- Large overlapping element communities → redundancy.
- Small, distinct communities → independent conceptual clusters.

Chapter 57
Set-Level Community Analysis

Abstract

At the set level, communities correspond to groups of categories that share many elements, collapsing into broader meta-groups. Community detection identifies whether the ontology effectively has fewer categories than defined [10, 5].

57.1 Mathematical formulation

Adjacency:

$$A_s(i,j) = |S_i \cap S_j|.$$

Apply modularity optimization on A_s.

57.2 Pseudocode

```
\Procedure{SetCommunities}{Sets}
  for each pair (i,j):
    A_s[i,j] = |S_i     S_j|
  Build graph
  Detect communities
  return partition of sets
\EndProcedure
```

57.3 Python illustration

```python
def set_communities(ontology_sets):
    n = len(ontology_sets)
    G = nx.Graph()
    for i in range(n):
        for j in range(i+1, n):
            w = len(ontology_sets[i] & ontology_sets[j])
            if w > 0:
                G.add_edge(i, j, weight=w)
    comms = community.greedy_modularity_communities(G, weight="weight")
    return comms
```

57.4 Analysis

- Communities of sets reveal redundancy and category collapse.
- High modularity ⇒ ontology divides into stable clusters.
- Low modularity ⇒ ontology resembles one “ball of mud.”

Chapter 58
Interrelationships: Element vs. Set Communities

Abstract

This chapter examines the relationship between element-level and set-level communities. Together, they reveal whether redundancy originates in elements co-occurring across sets or in sets overlapping heavily with each other [**?**].

58.1 Conceptual relationship

- Element communities: redundancy in features.
- Set communities: redundancy in categories.
- Comparing them clarifies the locus of ontological collapse.

58.2 Resolution

- If element communities are strong but set communities weak → redundancy originates in elements.
- If set communities are strong but element communities weak → redundancy originates in categories.
- If both are strong → systemic collapse.
- If both are weak → independence.

Volume VII

Ontology Engineering / Philosophy of Science

Part XV
Taxonomic Collapse Index

Chapter 59
Taxonomic Collapse Index (General)

Abstract

The Taxonomic Collapse Index (TCI) is a composite metric that integrates redundancy, overlap, and independence into a single diagnostic score of ontological health. It measures the extent to which categories lose their distinctness, collapsing into an undifferentiated whole. This chapter introduces TCI, its conceptual foundations, and mathematical formalism.

59.1 Introduction

Individual metrics such as Jaccard similarity, Redundancy Ratio, and Axis Independence provide partial insights into ontology structure. TCI unifies these signals into a holistic indicator of taxonomic collapse, enabling automated detection of ontological failure.

59.2 Historical Perspective

The problem of taxonomic collapse dates back to medieval scholastic classifications, where overlapping categories blurred conceptual clarity [1]. In modern science, Kuhn highlighted the breakdown of taxonomies as a hallmark of paradigm crisis [2]. Contemporary ontology evaluation emphasizes composite indices that integrate multiple measures [3]. TCI inherits this tradition by offering a meta-metric of structural coherence.

59.3 Conceptual Foundations

TCI measures the extent to which an ontology approaches the state of total overlap (collapse). It integrates measures of redundancy (RR), overlap (OI), and lack of independence (AIS).

59.4 Mathematical Formalism

$$TCI = \alpha \cdot RR + \beta \cdot OI + \gamma \cdot (1 - AIS)$$

with weights $\alpha, \beta, \gamma \geq 0$ normalized to $\alpha + \beta + \gamma = 1$.

59.5 Pseudocode

```
Input: Ontology sets
Compute RR, OI, AIS
TCI =   *RR +   *OI +   *(1 - AIS)
Output TCI
```

59.6 Python Algorithm

```python
def taxonomic_collapse_index(rr, oi, ais, alpha=0.33, beta=0.33, gamma=0.34):
    return alpha*rr + beta*oi + gamma*(1 - ais)
```

59.7 Analysis and Interpretation

- $TCI \approx 0$: healthy ontology, strong boundaries. - $TCI \approx 1$: collapsed ontology, indistinguishable categories.

59.8 Significance

As a composite measure, TCI consolidates multiple signals into a single diagnostic of collapse.

59.9 Conclusion

TCI is essential for automated ontology evaluation, capturing failure modes beyond individual metrics.

References

References

[1] Aquinas, T. (1274). *Summa Theologica*.
[2] Kuhn, T. (1962). *The Structure of Scientific Revolutions*. University of Chicago Press.
[3] Brank, J., Grobelnik, M., & Mladenić, D. (2005). A survey of ontology evaluation techniques. In *Proceedings of the Conference on Data Mining and Data Warehouses (SiKDD)*.

Chapter 60
Taxonomic Collapse Index: Element-Level Analysis

Abstract

Element-level TCI assesses collapse by analyzing how specific elements contribute to redundancy, overlap, and dependence. It identifies micro-drivers of structural failure in an ontology.

60.1 Introduction

At the element level, collapse may be caused by a handful of overstretched or ambiguous elements. Element-level TCI quantifies these local drivers of collapse.

60.2 Historical Perspective

The issue of "ambiguous terms" appears in Aristotle's Categories [1]. In modern classification, Garfield observed that "hub terms" drove structural redundancy across disciplines [2]. Element-level TCI continues this analysis in computational form.

60.3 Conceptual Foundations

For each element e, compute its contributions to RR, OI, and AIS, then aggregate them into $TCI_{element}(e)$.

60.4 Mathematical Formalism

$$TCI_{element}(e) = \alpha \cdot RR(e) + \beta \cdot OI(e) + \gamma \cdot (1 - AIS(e))$$

60.5 Pseudocode

```
For each element e in U:
   compute RR(e), OI(e), AIS(e)
   TCI_element(e) =   *RR(e) +   *OI(e) +   *(1 - AIS(e))
Output dictionary of TCI_element values
```

60.6 Python Algorithm

```
def element_level_tci(rr_e, oi_e, ais_e, alpha=0.33, beta=0.33, gamma=0.34):
    return {e: alpha*rr_e[e] + beta*oi_e[e] + gamma*(1 - ais_e[e])
            for e in rr_e}
```

60.7 Analysis

- High $TCI_{element}$: element is a collapse driver. - Low $TCI_{element}$: element contributes little to collapse.

60.8 Significance

This identifies "fault lines" in the ontology at the level of specific terms.

60.9 Conclusion

Element-level TCI localizes sources of collapse, guiding fine-grained evaluation.

References

References

[1] Aristotle. (350 BCE). *Categories*.
[2] Garfield, E. (1979). *Citation Indexing: Its Theory and Application*. Wiley.

Chapter 61
Taxonomic Collapse Index: Set-Level Analysis

Abstract

Set-level TCI evaluates collapse across categories, assessing whether entire sets have lost distinctness due to redundancy and correlation.

61.1 Introduction

Whereas element-level analysis focuses on micro-drivers, set-level TCI provides a macro-level picture of structural collapse.

61.2 Historical Perspective

Library science confronted the issue of redundant subject headings in classification [1]. Similar concerns appear in biological taxonomy, where redundant genera indicated collapse of categories. Set-level TCI formalizes this with computational precision.

61.3 Conceptual Foundations

For each pair of sets, compute overlap and correlation. Aggregate these measures into a composite collapse index.

61.4 Mathematical Formalism

$$TCI_{set} = \alpha \cdot RR_{set} + \beta \cdot OI_{set} + \gamma \cdot (1 - AIS_{set})$$

61.5 Pseudocode

```
Compute RR_set, OI_set, AIS_set
TCI_set =   *RR_set +   *OI_set +   *(1 - AIS_set)
```

61.6 Python Algorithm

```
def set_level_tci(rr_set, oi_set, ais_set, alpha=0.33, beta=0.33, gamma=0.34):
    return alpha*rr_set + beta*oi_set + gamma*(1 - ais_set)
```

61.7 Analysis

- High TCI_{set}: sets collapsed into redundancy. - Low TCI_{set}: strong boundaries preserved.

61.8 Significance

This metric diagnoses systemic category collapse.

61.9 Conclusion

Set-level TCI captures macro-structural failures of ontology.

References

References

[1] Beghtol, C. (1986). Semantic validity: Concepts of warrant in bibliographic classification systems. *Library Resources & Technical Services*, 30(2), 109–125.

Chapter 62
Taxonomic Collapse Index: Interrelationships Between Element-Level and Set-Level

Abstract

This chapter explores the relationship between element-level and set-level TCI, emphasizing scenarios where their values diverge and the interpretive consequences of such discrepancies.

62.1 Introduction

Element-level and set-level collapse are related but not identical. Comparing them provides a multi-scale picture of ontological health.

62.2 Historical Perspective

The micro–macro problem in social science—whether individual behavior explains structural collapse—has parallels in ontology analysis [1]. Bibliometric studies similarly distinguish between term-level redundancy and field-level collapse [2].

62.3 Conceptual Foundations

- Element-level TCI: highlights ambiguous or overstretched elements. - Set-level TCI: highlights systemic collapse of categories.

62.4 Analysis

- Both high: ontology globally collapsed. - Element high, set low: few elements drive redundancy. - Set high, element low: structural duplication across sets.

62.5 Resolution

The comparison resolves whether collapse is localized or systemic.

62.6 Conclusion

Interrelating element- and set-level TCI provides a holistic diagnostic of ontological collapse.

References

References

[1] Durkheim, E. (1895). *The Rules of Sociological Method*. Paris.
[2] Soergel, D. (1985). *Organizing Information: Principles of Database and Retrieval Systems*. Academic Press.

Part XVI
Conclusion

Chapter 63
Conclusion: Integrating Ontological Diagnostics

Abstract

This chapter integrates the multiple lines of analysis developed in previous chapters, showing how set-theoretic, topological, statistical, and network-theoretic diagnostics combine to evaluate ontologies. Each method provides a partial perspective; together they form a coherent framework for identifying redundancy, overlap, collapse, or independence in structured knowledge systems [1, 2, 3, 4, 5].

63.1 Synthesis of diagnostics

The following table summarizes each diagnostic, its input, its focus, and what it reveals about ontology structure:

Method	Input	Focus	Reveals
Intersection counts	Set incidence	Raw overlaps between sets	Intensity of sharing
Jaccard index	Pairwise sets	Normalized overlap	Relative similarity, scale-independent
Betti curves	Overlap graph	Topological persistence	Cycles, higher-order redundancy
PCA (scree)	Covariance matrix	Eigenvalue spectrum	Effective dimensionality
PCA (loadings)	Eigenvectors	Variable contributions	Semantic meaning of dimensions
Membership histogram	Elements/sets	Frequency distribution	Redundancy concentration
Community detection	Set graph	Modularity structure	Collapse into meta-groups

Table 63.1 Summary of diagnostics in ontological analysis.

63.2 Complementarity

The complementarity of methods is summarized in Table 63.2.

Pair of Methods	How They Complement	When to Use Together
Intersection counts + Jaccard	Raw vs. normalized overlap	To detect both absolute and relative redundancy
PCA (scree) + PCA (loadings)	Variance vs. semantic content	To identify both number and meaning of latent dimensions
Betti curves + Membership histogram	Topology vs. distribution	To link higher-order structure to redundancy patterns
Community detection + PCA	Graph clusters vs. linear decomposition	To confirm modularity vs. continuous variance collapse

Table 63.2 Complementarity among diagnostics.

63.3 Resolution of disagreements

When different diagnostics diverge, their outputs can be aligned as shown in Table 63.3.

Diagnostic Pattern	Interpretation	Resolution
High Jaccard, flat Betti	Dense overlaps, no higher structure	Overlaps are real but shallow
Strong PCA collapse, flat histograms	Latent redundancy not visible in distribution	Trust PCA as more sensitive
Communities strong, PCA independence	Redundancy localized in modular clusters	Treat ontology as modular, not collapsed
Both flat (all)	Weak structure overall	Ontology is fragmented or invalid

Table 63.3 Resolution framework for conflicting diagnostics.

63.4 Conclusion

Ontological evaluation requires a suite of diagnostics. When they converge, the ontology's structural state is clear; when they diverge, resolution tables provide guidance. The integrated framework developed here demonstrates how combining set theory, topology, statistics, and network science enables a rigorous assessment of ontological validity.

References

[1] Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In S. Staab & R. Studer (Eds.), *Handbook on Ontologies*. Springer.

[2] Smith, B. (2008). Ontology (science). In *Formal Ontology in Information Systems*, IOS Press.

[3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. AMS.

[4] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

[5] Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577–8582.

Appendix A
Appendix: Master Synthesis Table

Method	Input / Focus	Reveals	Best Complement	Resolution of Disagreements
Intersection Counts	Raw set overlaps	Intensity of shared elements	Jaccard (scale-normalized)	If counts high but Jaccard low → redundancy limited to large sets
Jaccard Index	Normalized pairwise overlap	Relative similarity independent of size	Intersection counts	If Jaccard high but Betti flat → overlaps are shallow, not structured
Betti Curves	Overlap graph topology	Persistence of cycles / higher-order redundancy	Membership histograms	If Betti flat but hist skewed → redundancy exists but not topological
PCA (Scree)	Eigenvalue spectrum	Effective dimensionality	PCA (Loadings)	If Scree flat but Loadings strong → weak variance but semantic structure present
PCA (Loadings)	Eigenvectors scaled	Contribution of variables to components	PCA (Scree)	If Loadings weak but Scree strong → variance exists but lacks semantic clarity
Membership Histogram	Elements or sets	Redundancy distribution	Betti curves	If hist balanced but PCA collapse → hidden latent redundancy not seen in counts
Community Detection	Graph modularity	Collapse into meta-groups	PCA or histograms	If communities strong but PCA independent → modular redundancy not global

Table A.1 Master synthesis table summarizing diagnostics, complements, and resolution strategies in ontological analysis.

Appendix B
Appendix: Diagnostic Network Map
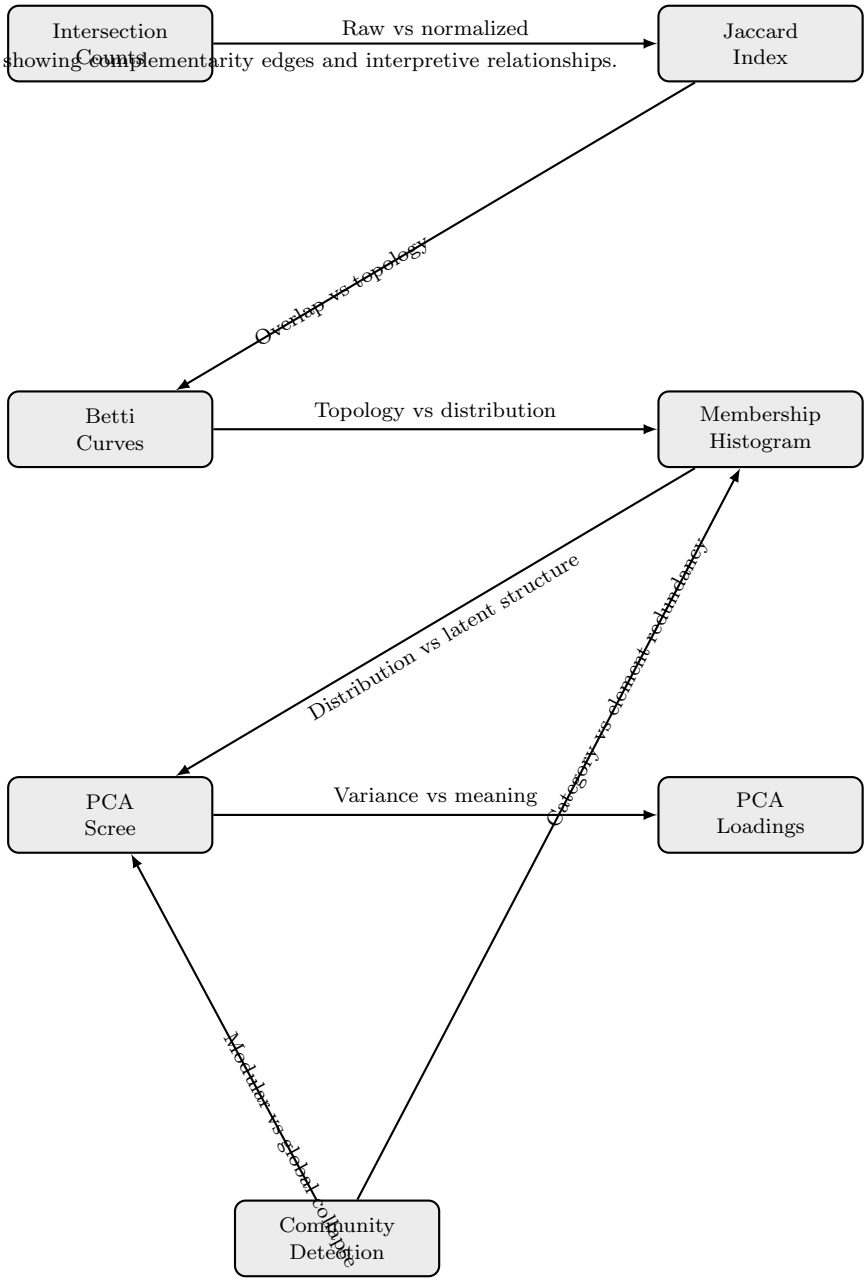
Figure: Network of diagnostic methods, showing complementarity edges and interpretive relationships.